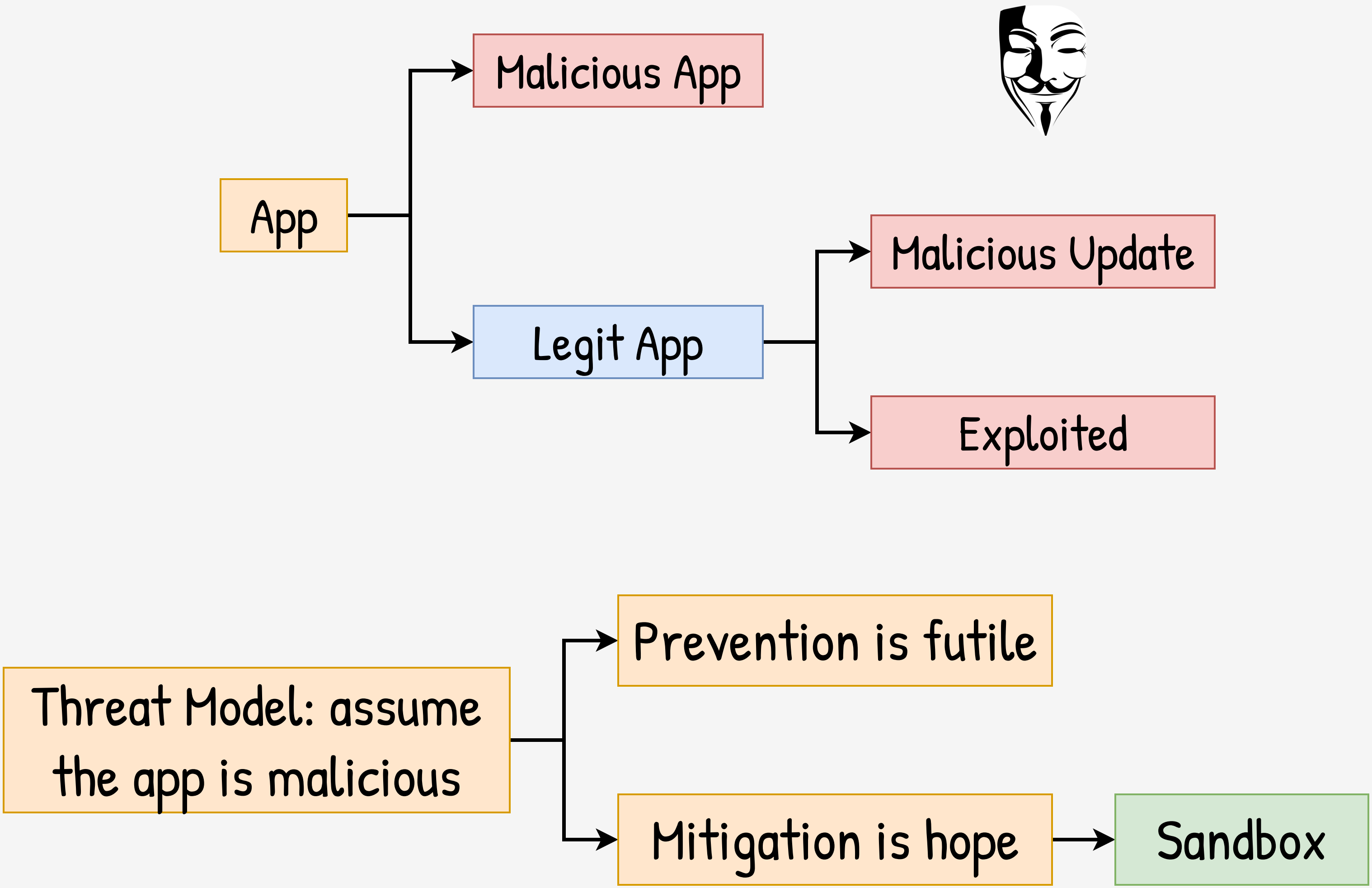
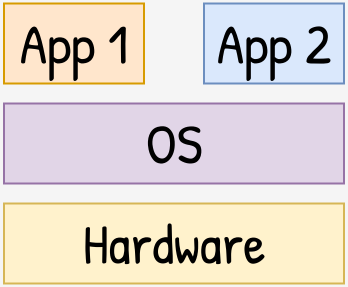
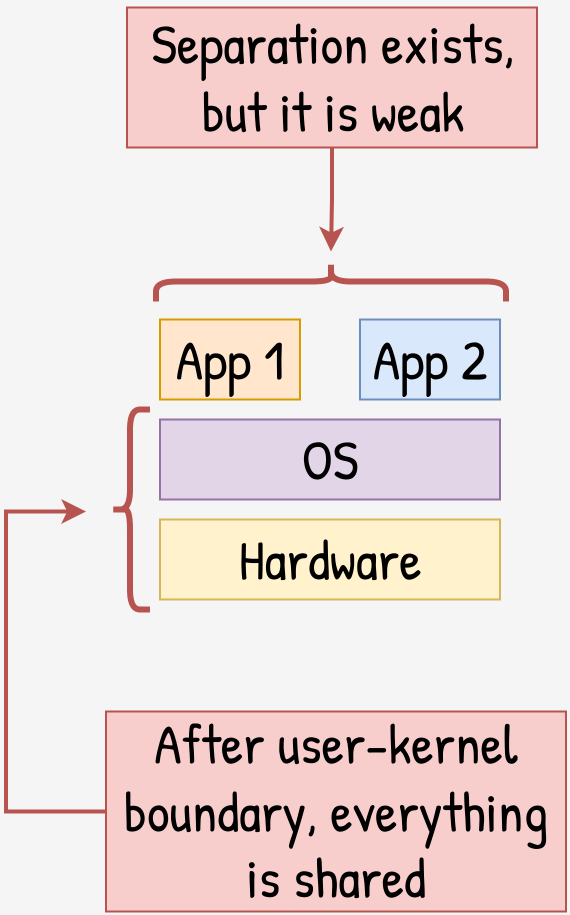
Sandbox at OS level: Seccomp, cgroups, capabilities, namespaces & mounts

Taller de Seguridad

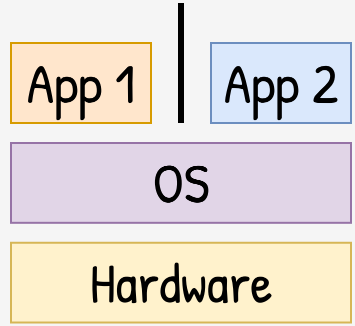
Ing. Martin Di Paola - Fiuba


App
Malicious App
Legit App
Malicious Update
Exploited
Threat Model: assume the app is malicious
Prevention is futile
Mitigation is hope
Sandbox

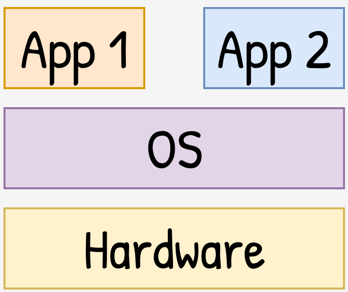

App 1
App 2
OS
Hardware

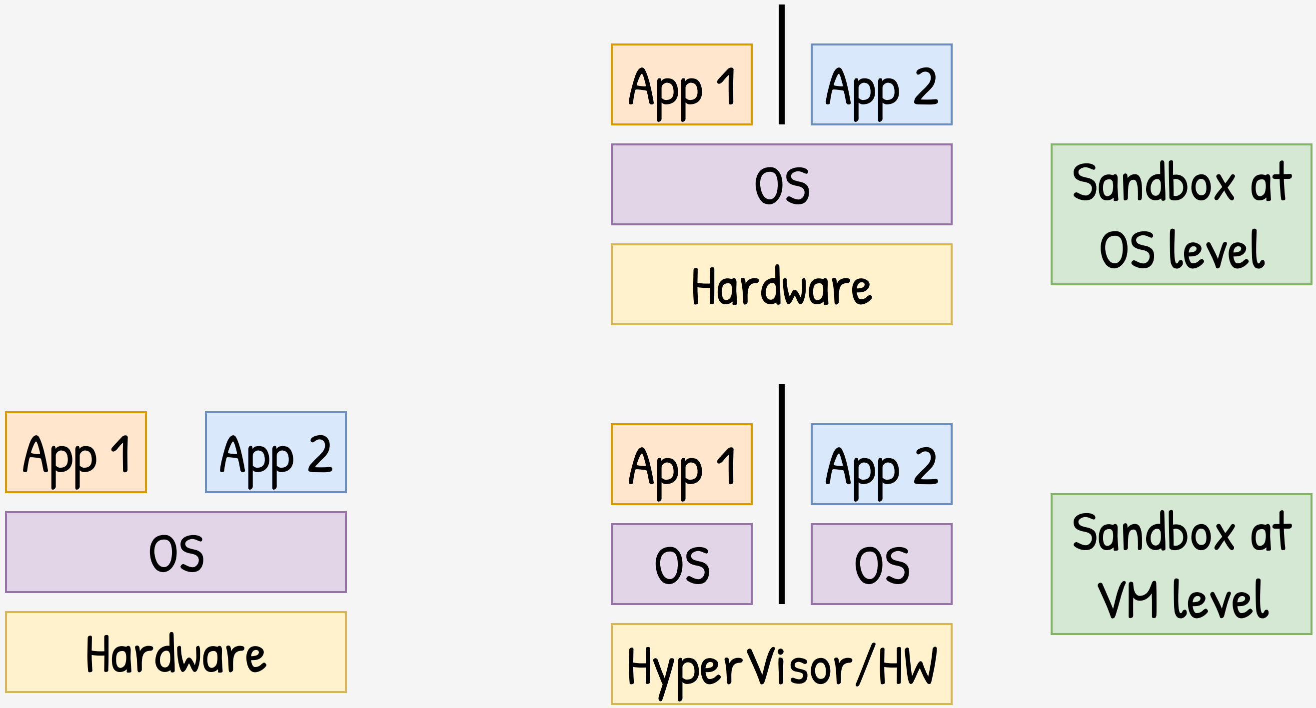


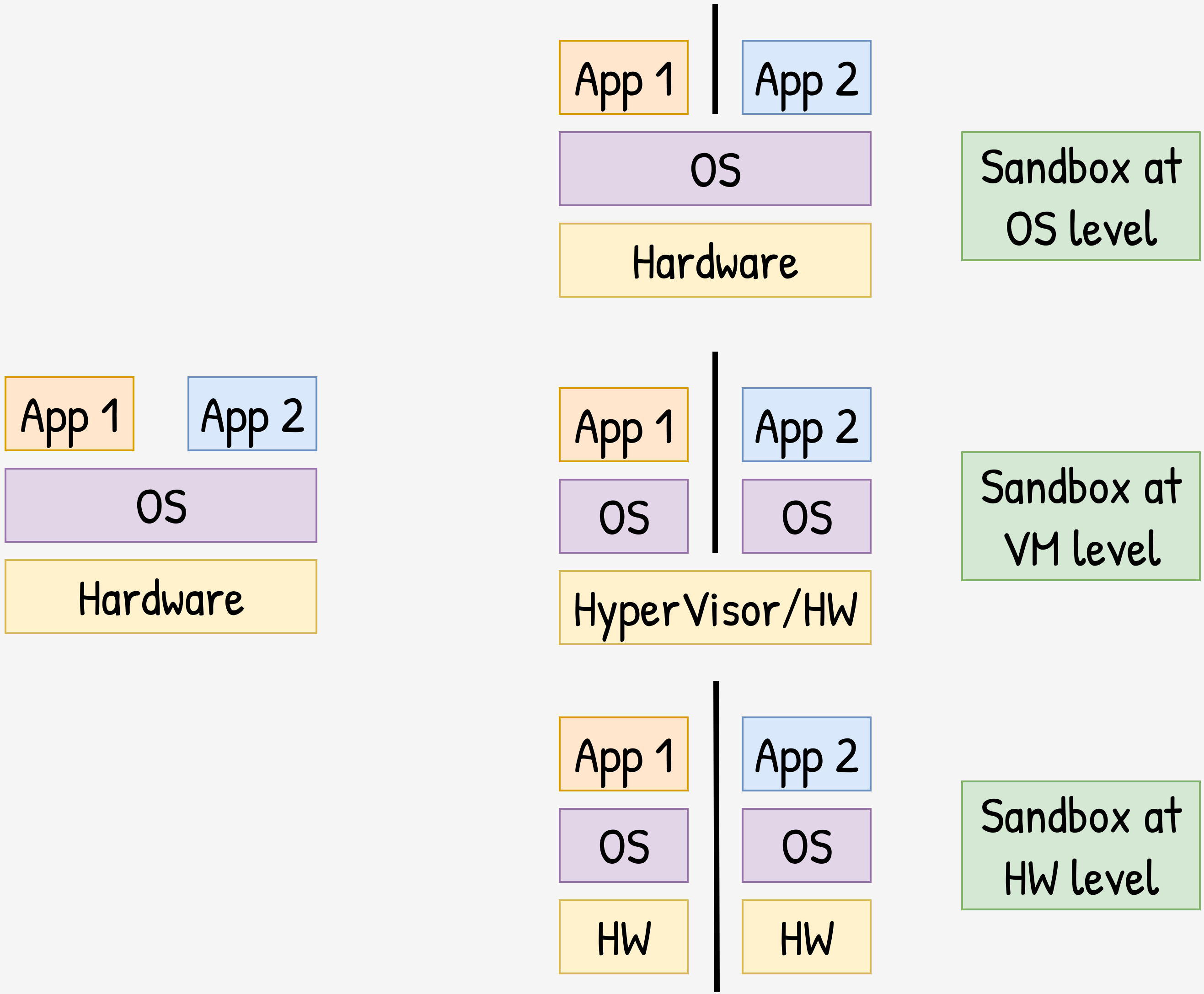
Separation exists,
but it is weak
App 1
App 2
OS
Hardware
After user-kernel
boundary, everything
is shared

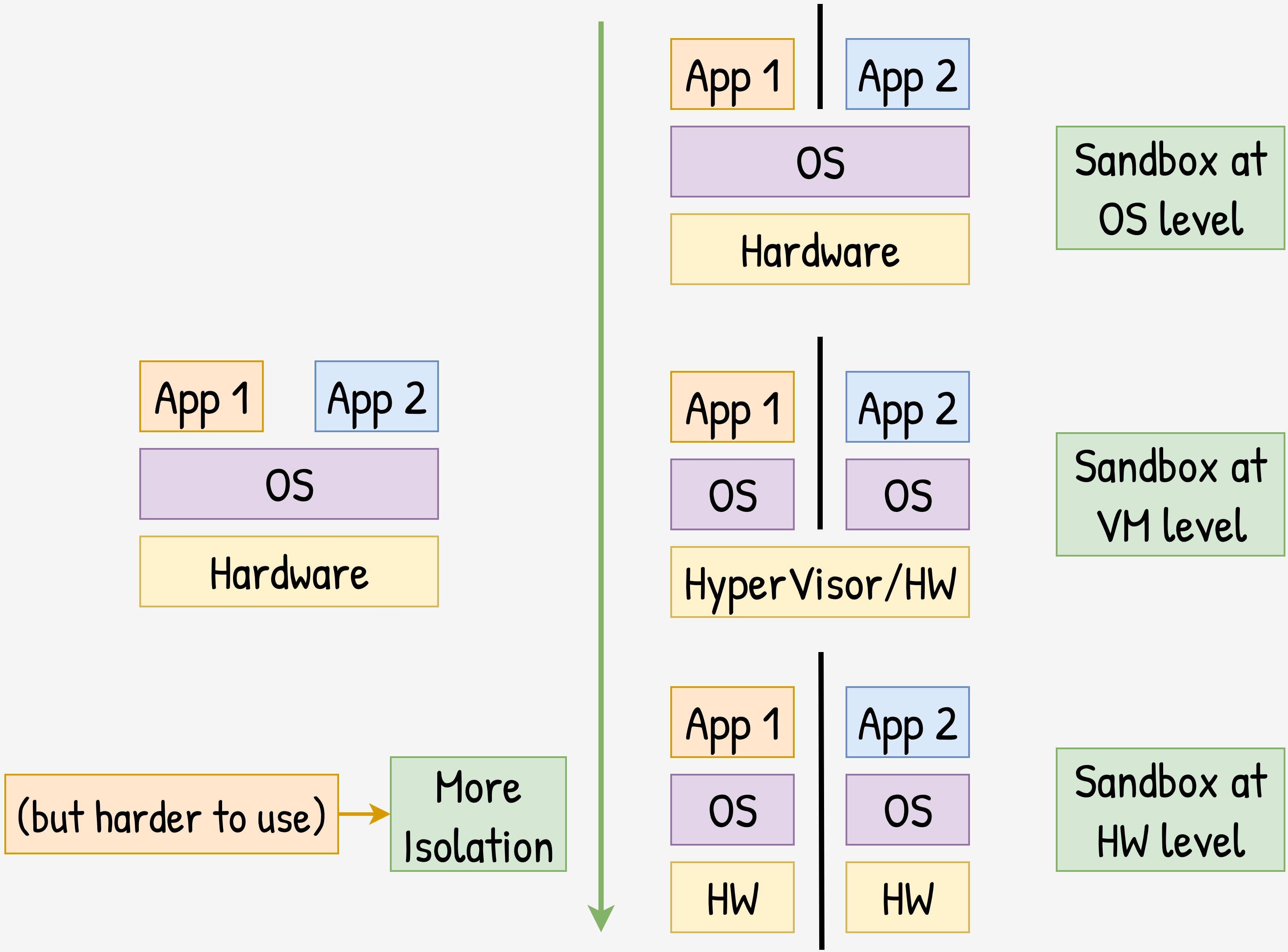


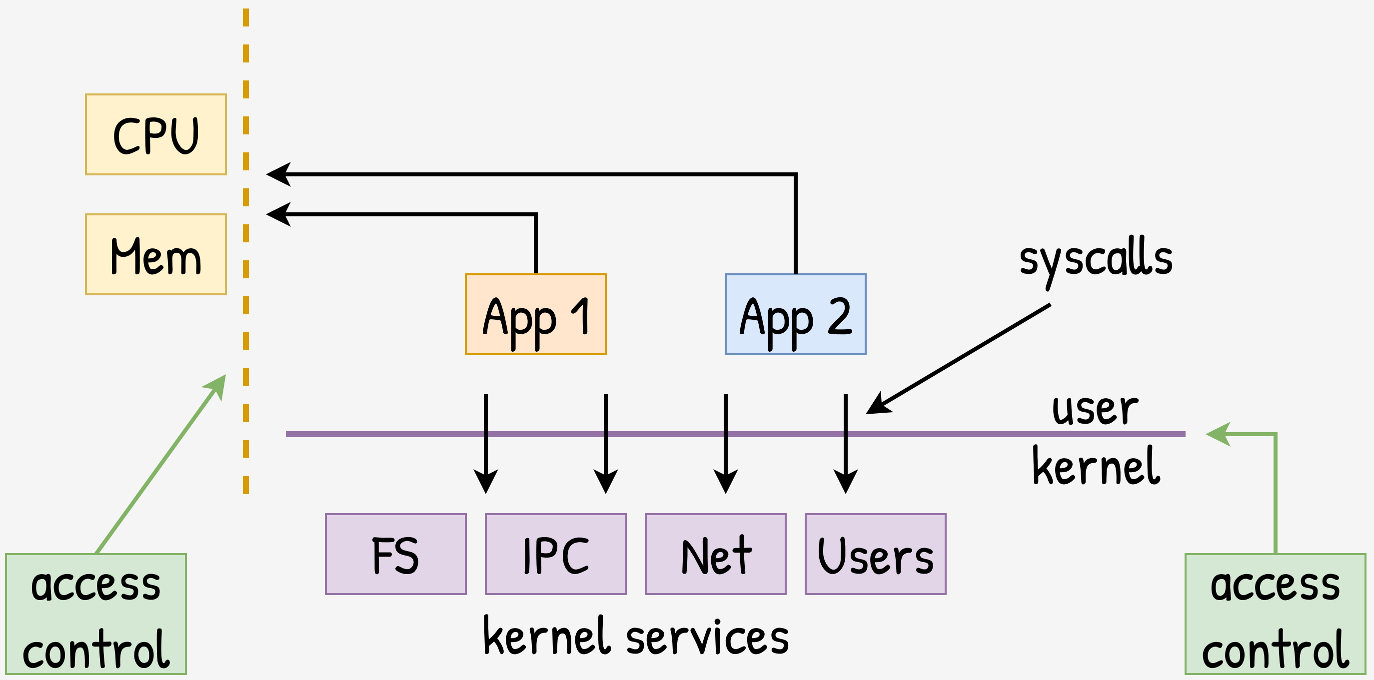
Sandbox at OS level

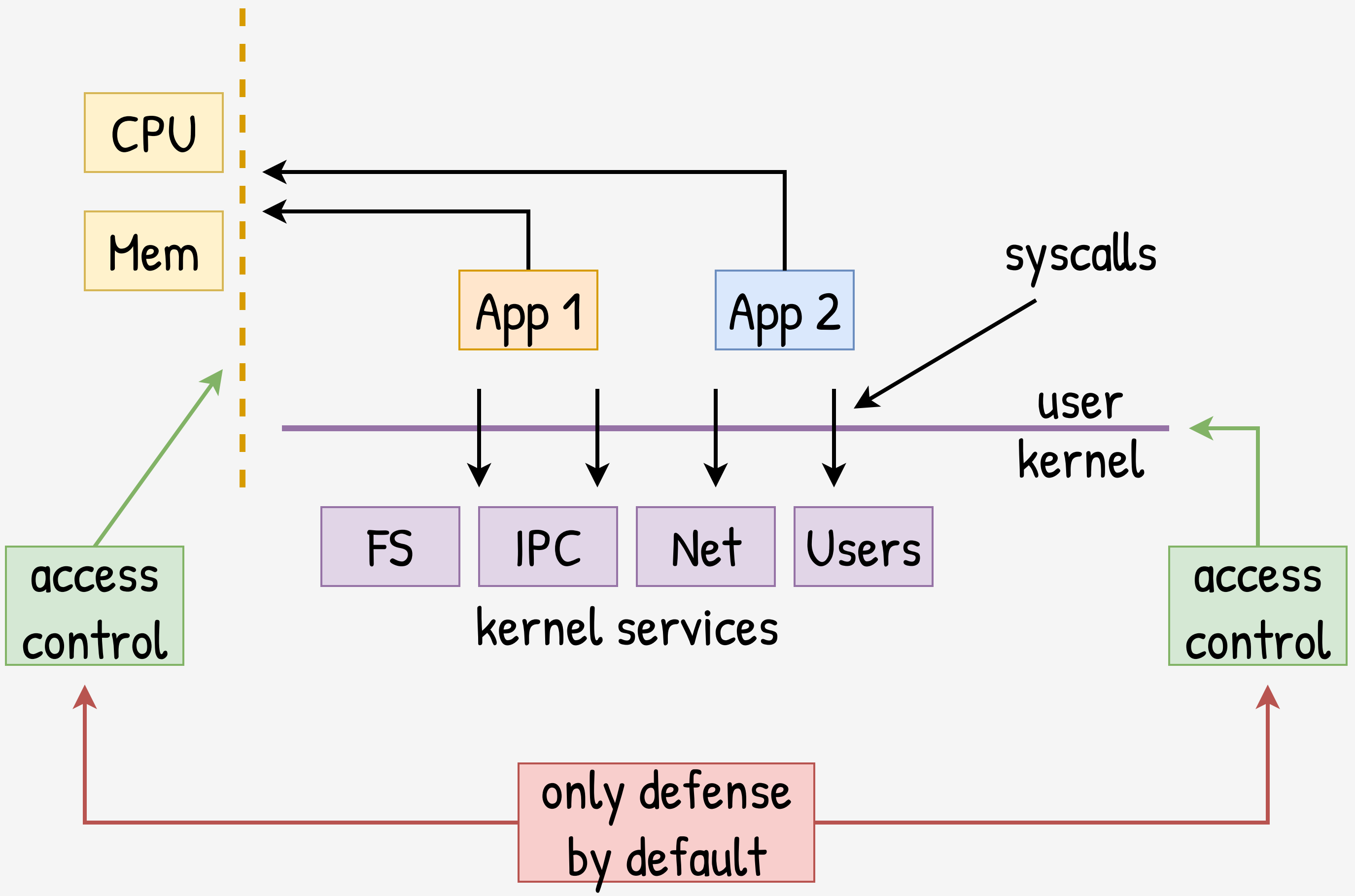



App 1
App 2
OS
Hardware
App 1
App 2
OS
Hardware
Sandbox at OS level
App 1
App 2
OS
OS
HyperVisor/HW
Sandbox at VM level

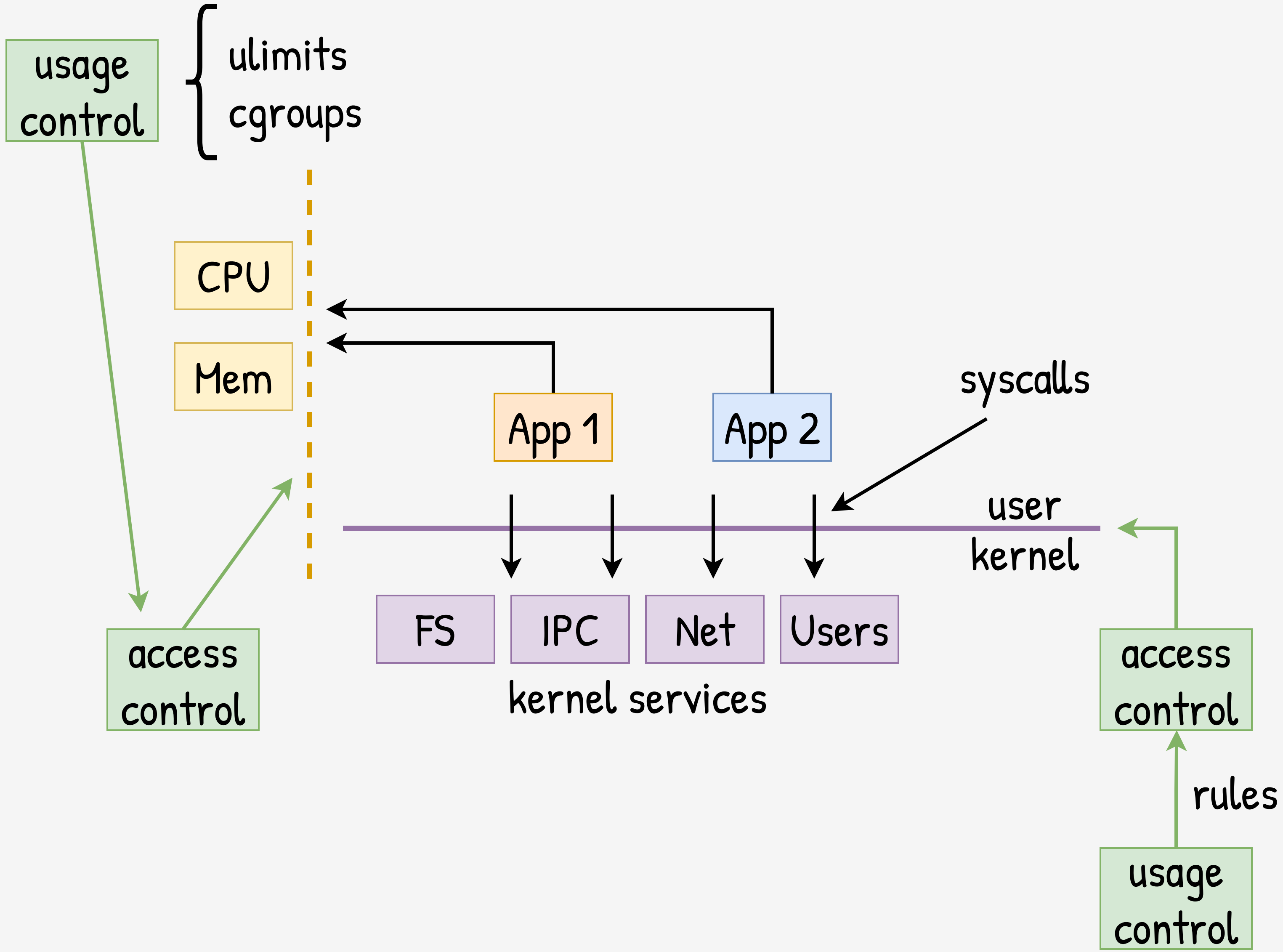

App 1
App 2
OS
Hardware
Sandbox at OS level
Sandbox at VM level
HyperVisor/HW
Sandbox at HW level
HW

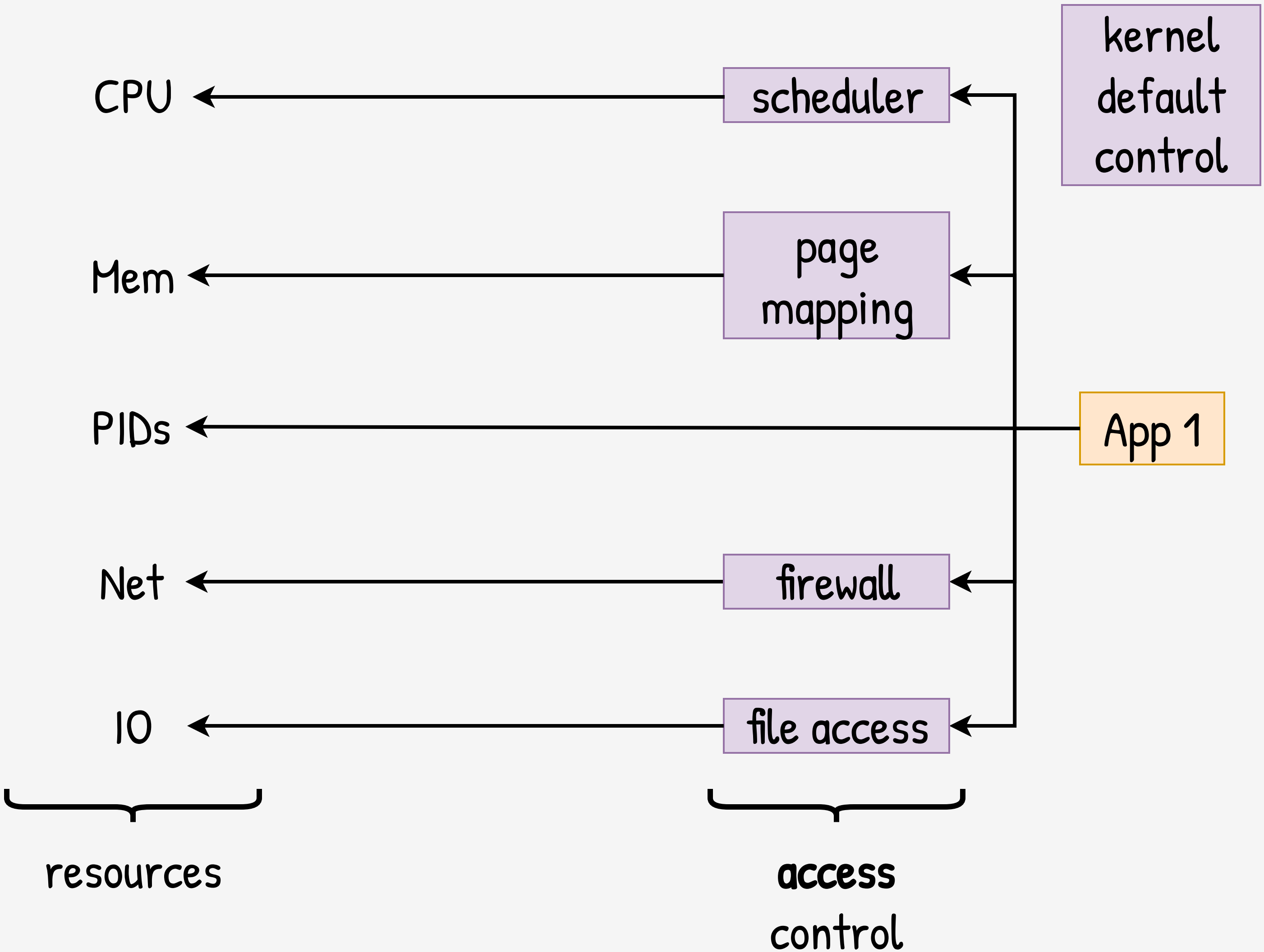

App 1
App 2
OS
Hardware
Sandbox at OS level
App 1
App 2
OS
OS
HyperVisor/HW
Sandbox at VM level
App 1
App 2
OS
OS
HW
HW
Sandbox at HW level
App 1
App 2
OS
Hardware
(but harder to use)
More Isolation

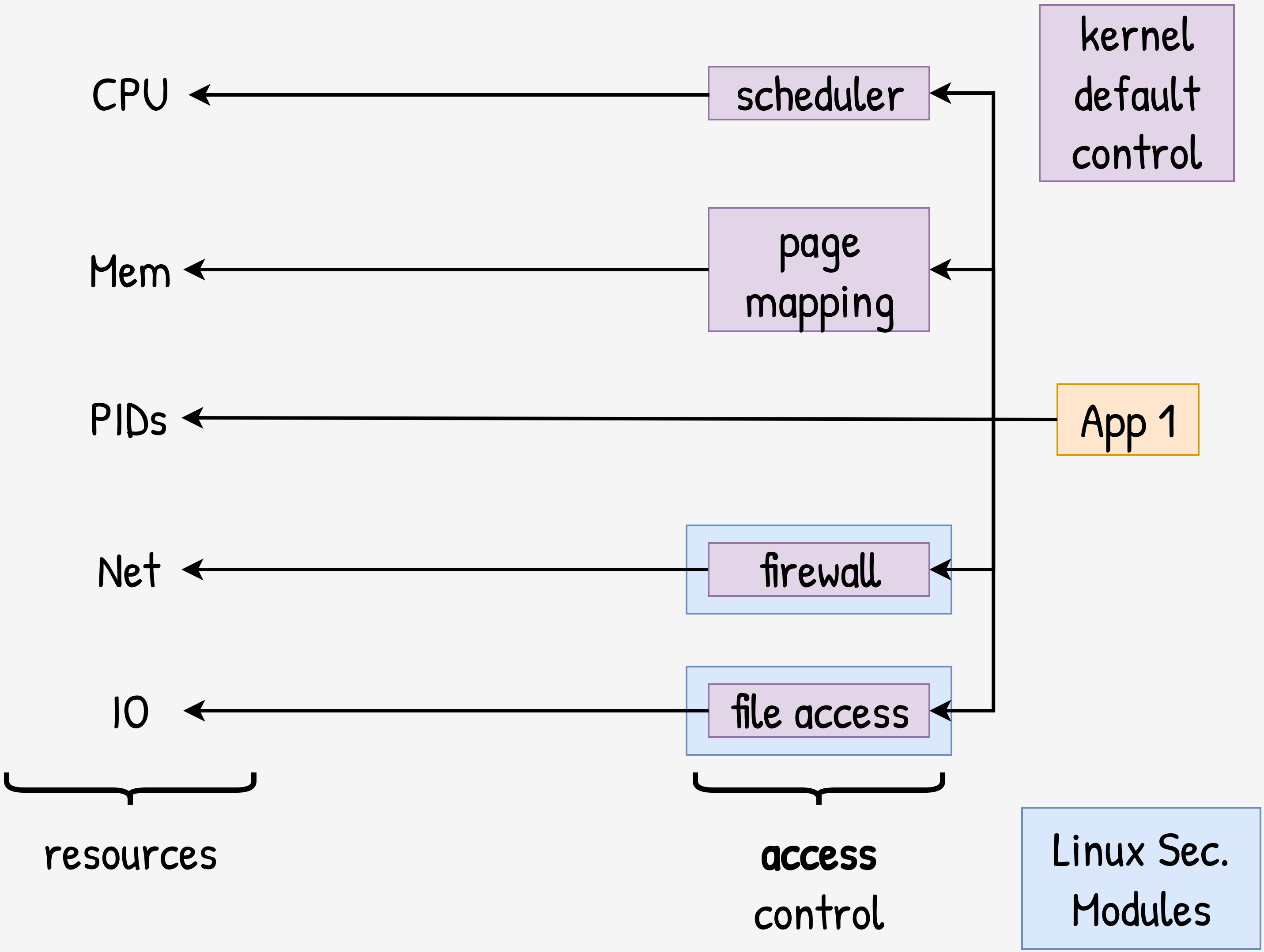

CPU
Mem
App 1
App 2
syscalls
user
kernel
FS
IPC
Net
Users
kernel services
access control
access control



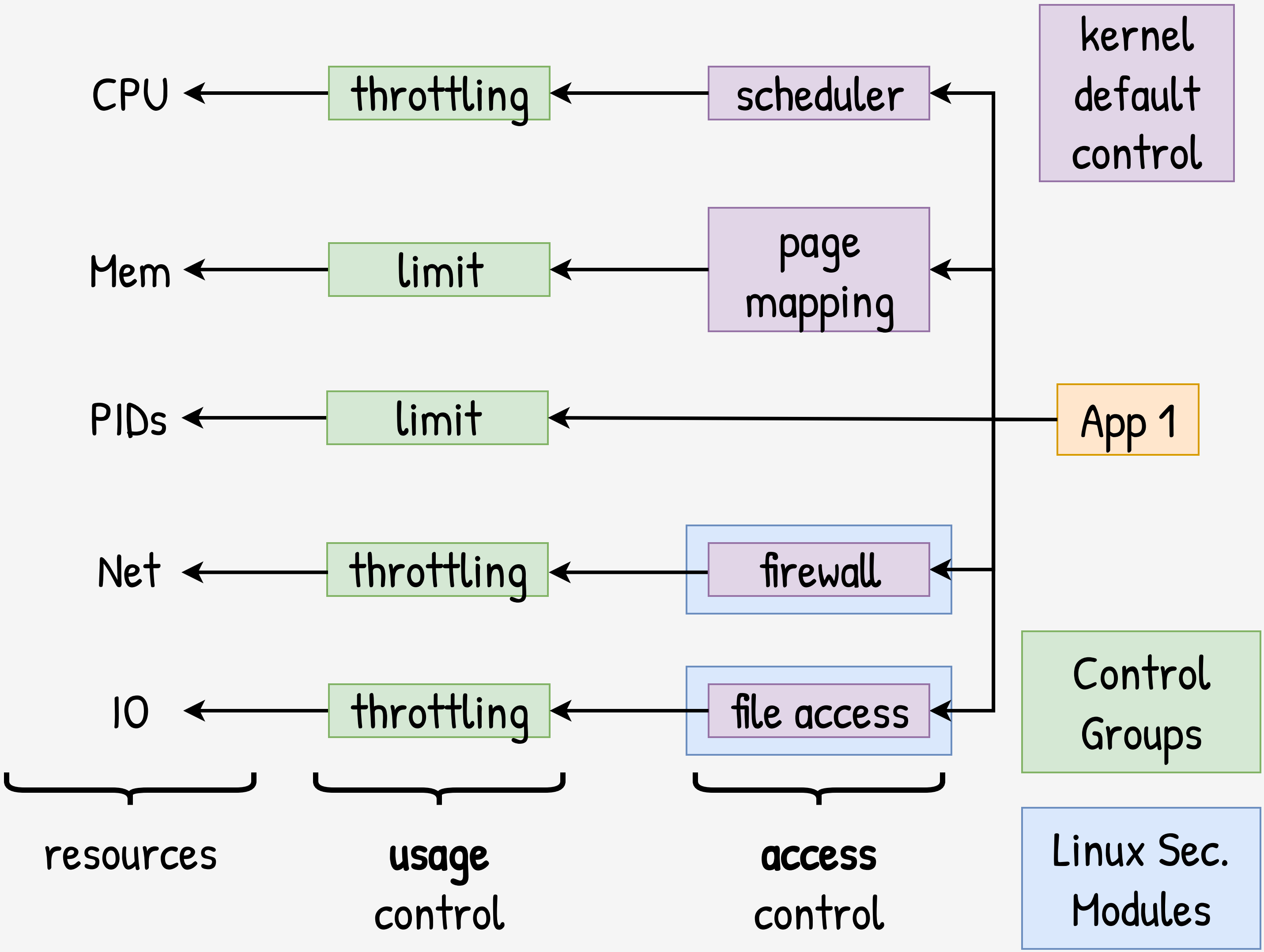
CPU
Mem
App 1
App 2
syscalls
user
kernel
FS
IPC
Net
Users
kernel services
access control
access control
only defense by default


usage control
ulimits
cgroups
CPU
Mem
App 1
App 2
syscalls
user
kernel
access control
FS
IPC
Net
Users
kernel services
access control
rules
usage control

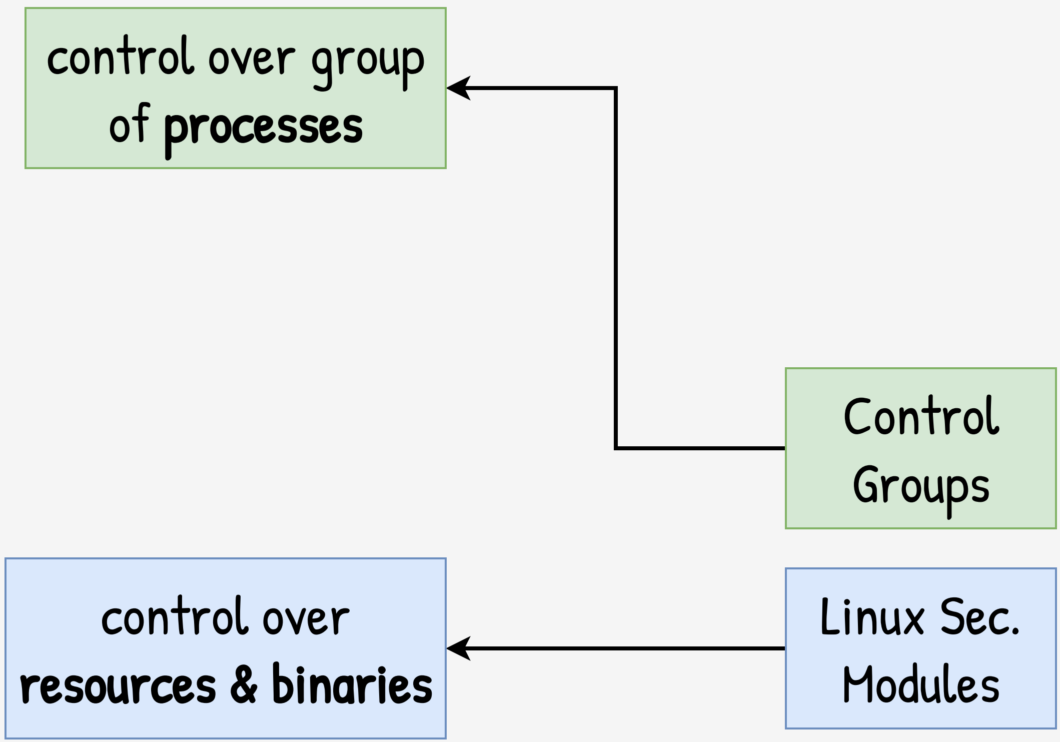

kernel
default
control
CPU
scheduler
Mem
page
mapping
PIDs
App 1
Net
firewall
IO
file access
resources
access
control

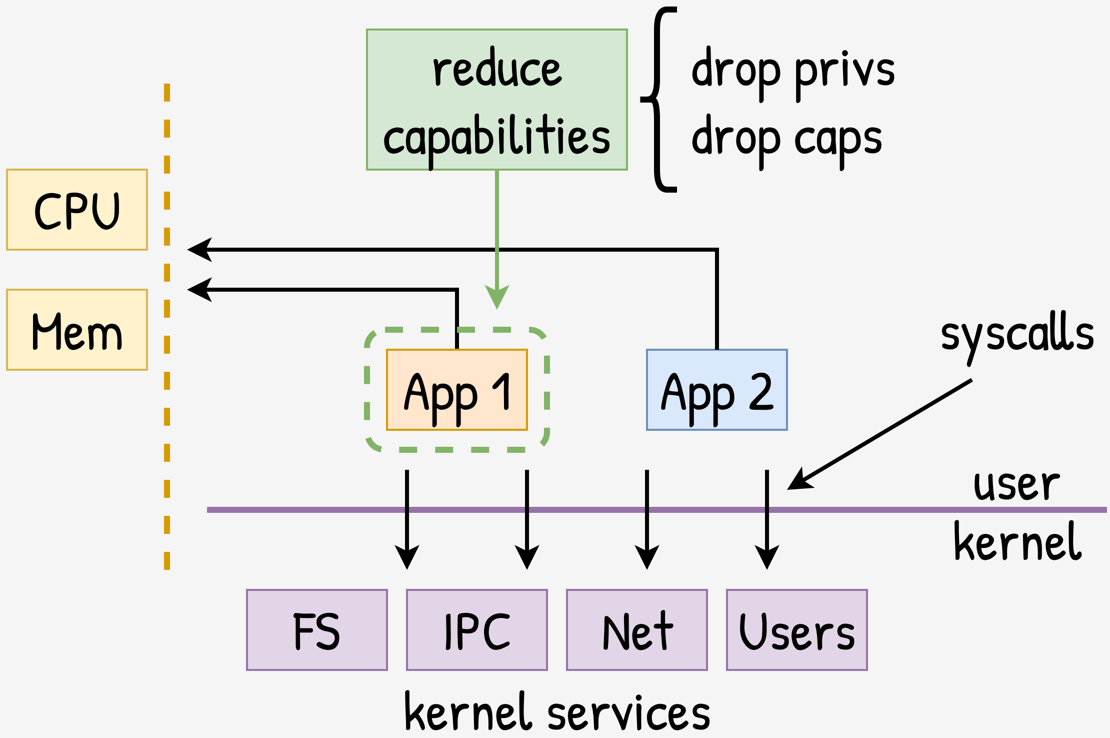


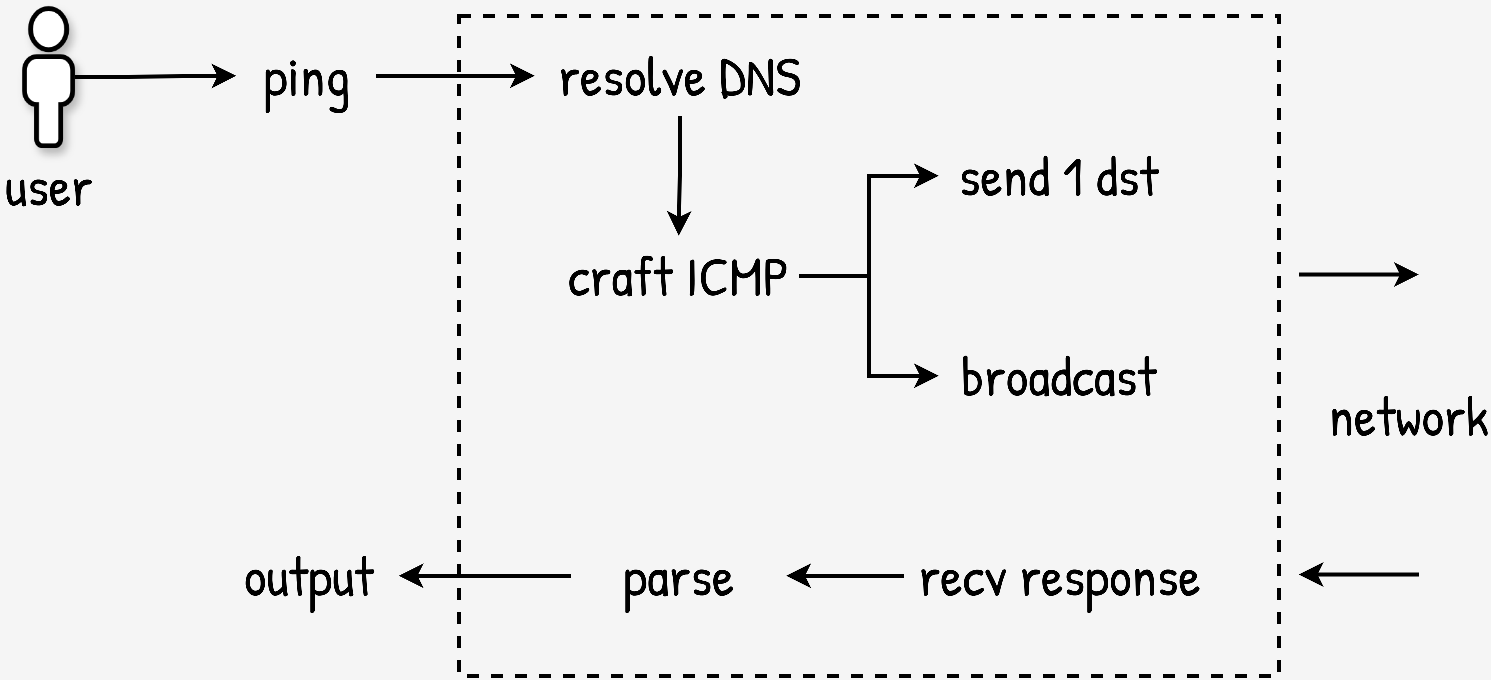
kernel default control
CPU
scheduler
Mem
page mapping
PIDs
App 1
Net
firewall
IO
file access
resources
access control
Linux Sec. Modules

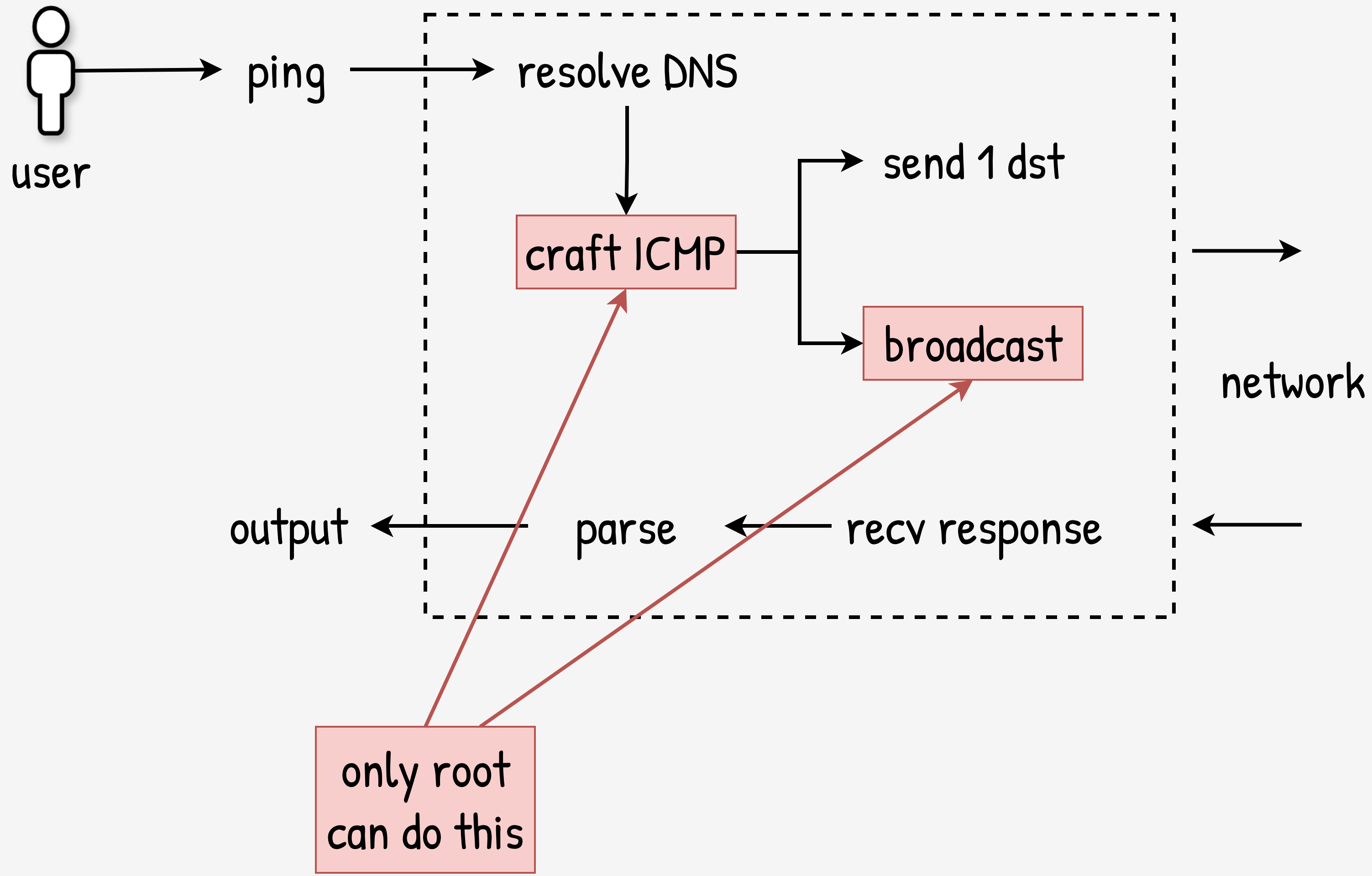


kernel default control
CPU
throttling
scheduler
Mem
limit
page mapping
PIDs
limit
App 1
Net
throttling
firewall
IO
throttling
file access
Control Groups
resources
usage control
access control
Linux Sec. Modules

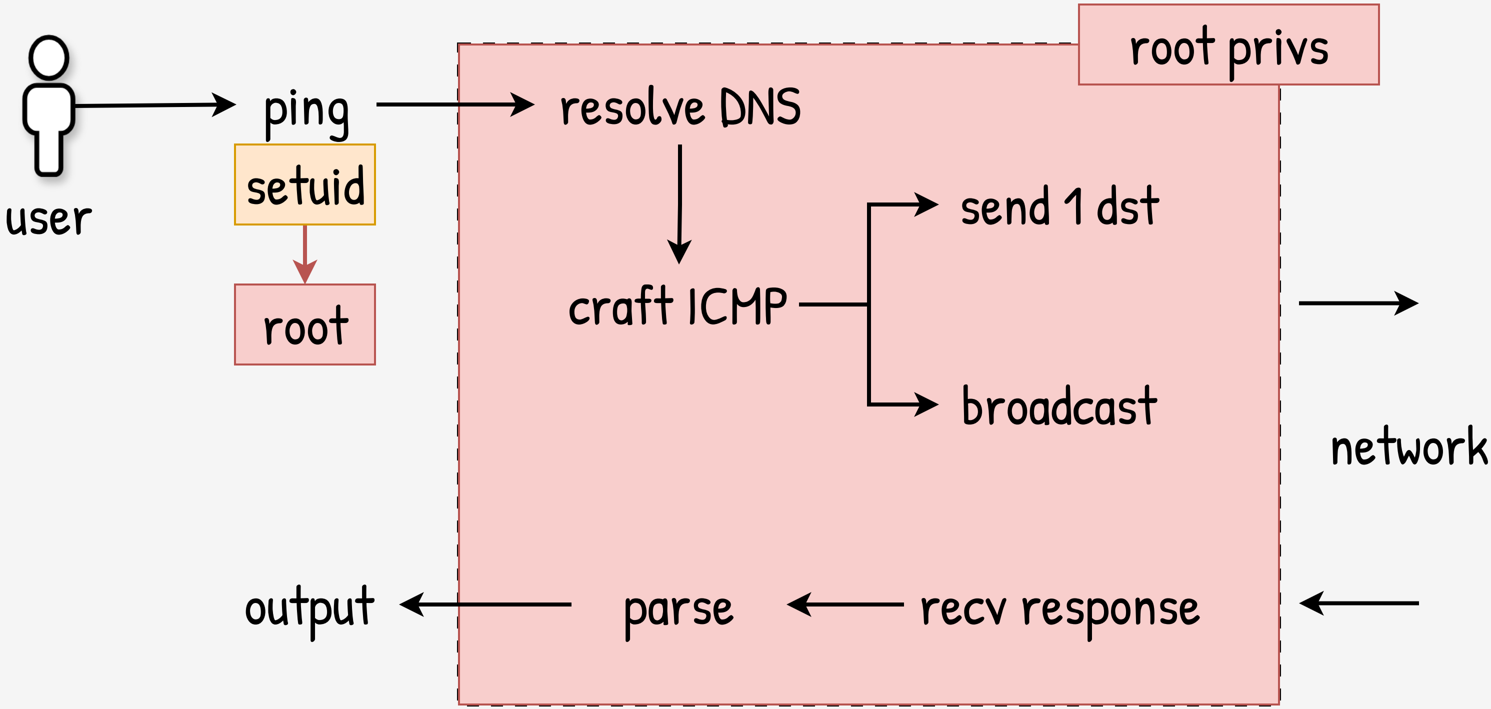

control over group of **processes**
Control Groups
control over **resources & binaries**
Linux Sec. Modules


reduce capabilities
drop privs
drop caps
CPU
Mem
App 1
App 2
syscalls
user
kernel
FS
IPC
Net
Users
kernel services

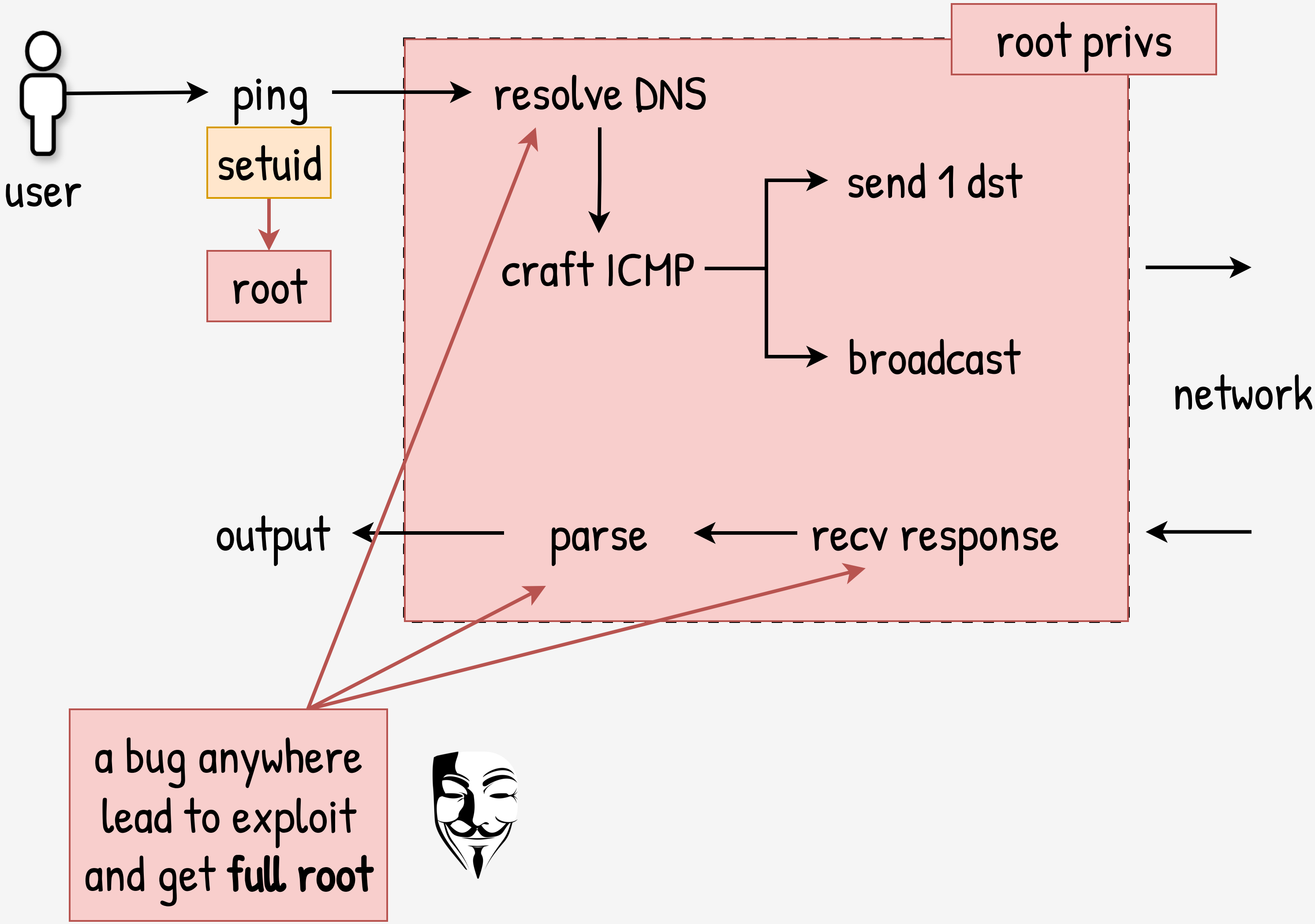

user
ping
resolve DNS
craft ICMP
send 1 dst
broadcast
network
recv response
parse
output

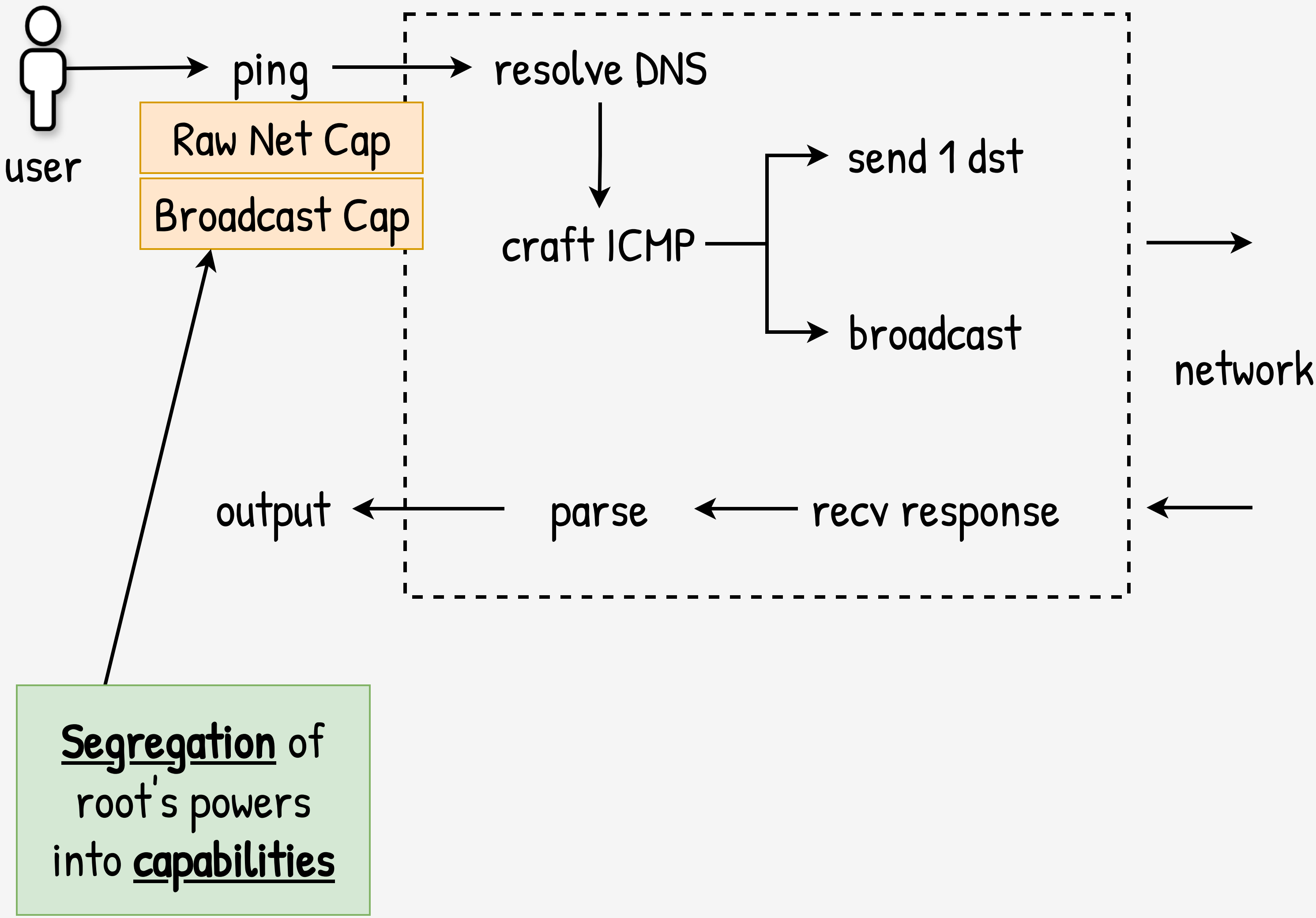


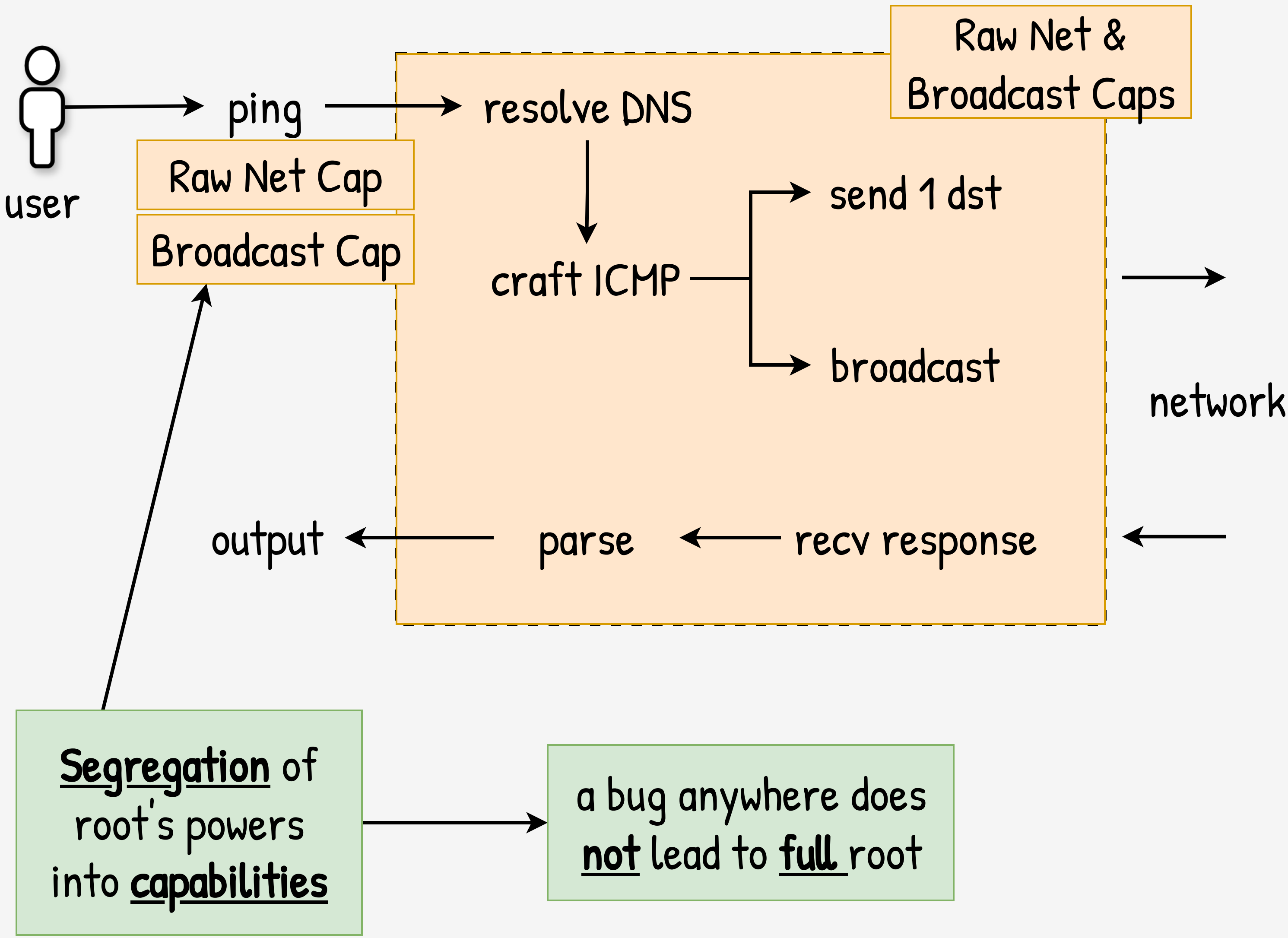
user
ping
resolve DNS
craft ICMP
send 1 dst
broadcast
network
output
parse
recv response
only root can do this

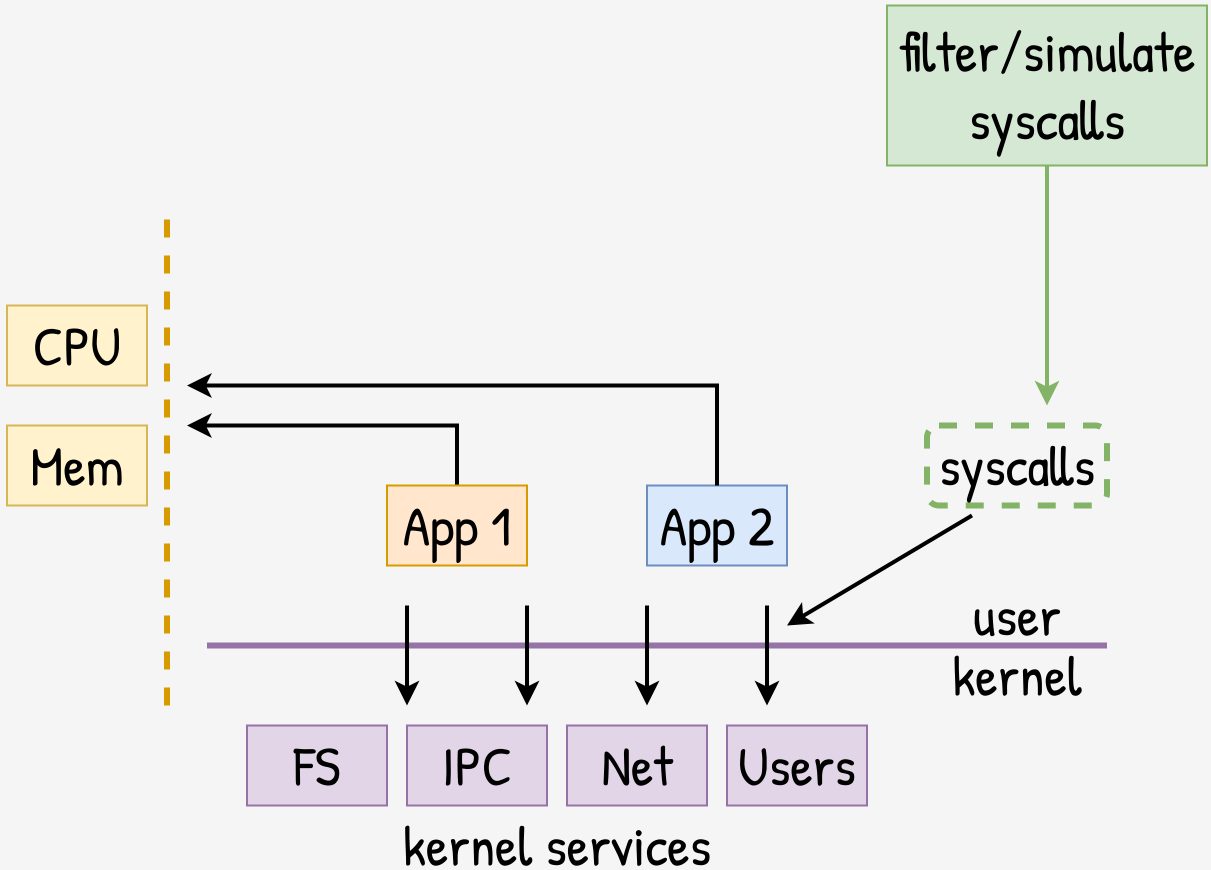


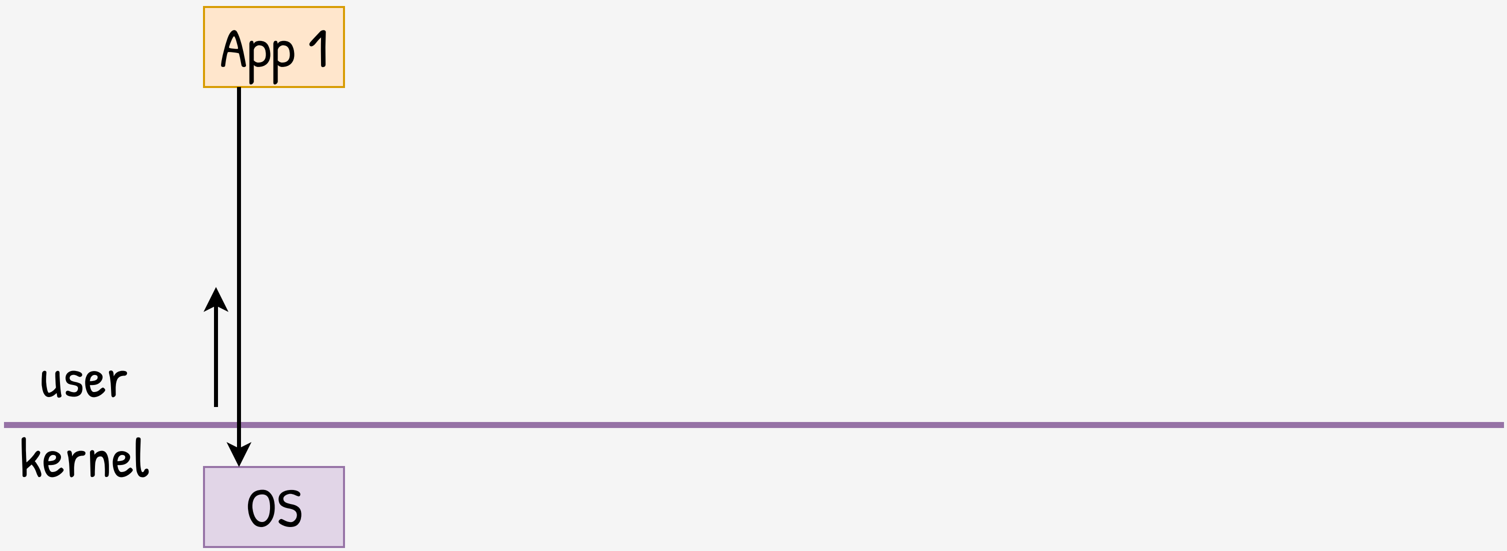
user
ping
setuid
root
root privs
resolve DNS
craft ICMP
send 1 dst
broadcast
network
output
parse
recv response

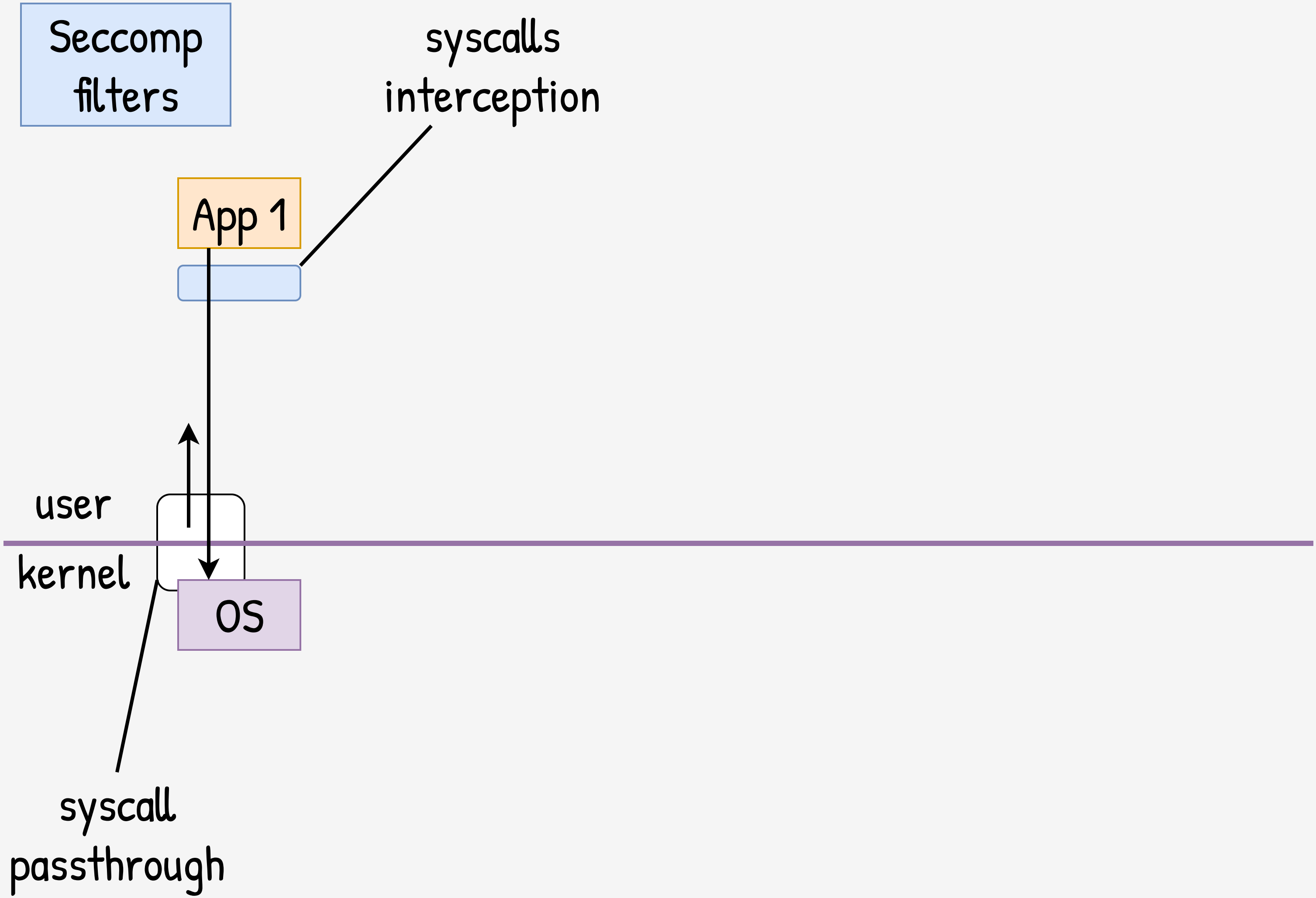

user
ping
setuid
root
root privs
resolve DNS
craft ICMP
send 1 dst
broadcast
network
output
parse
recv response
a bug anywhere
lead to exploit
and get **full root**

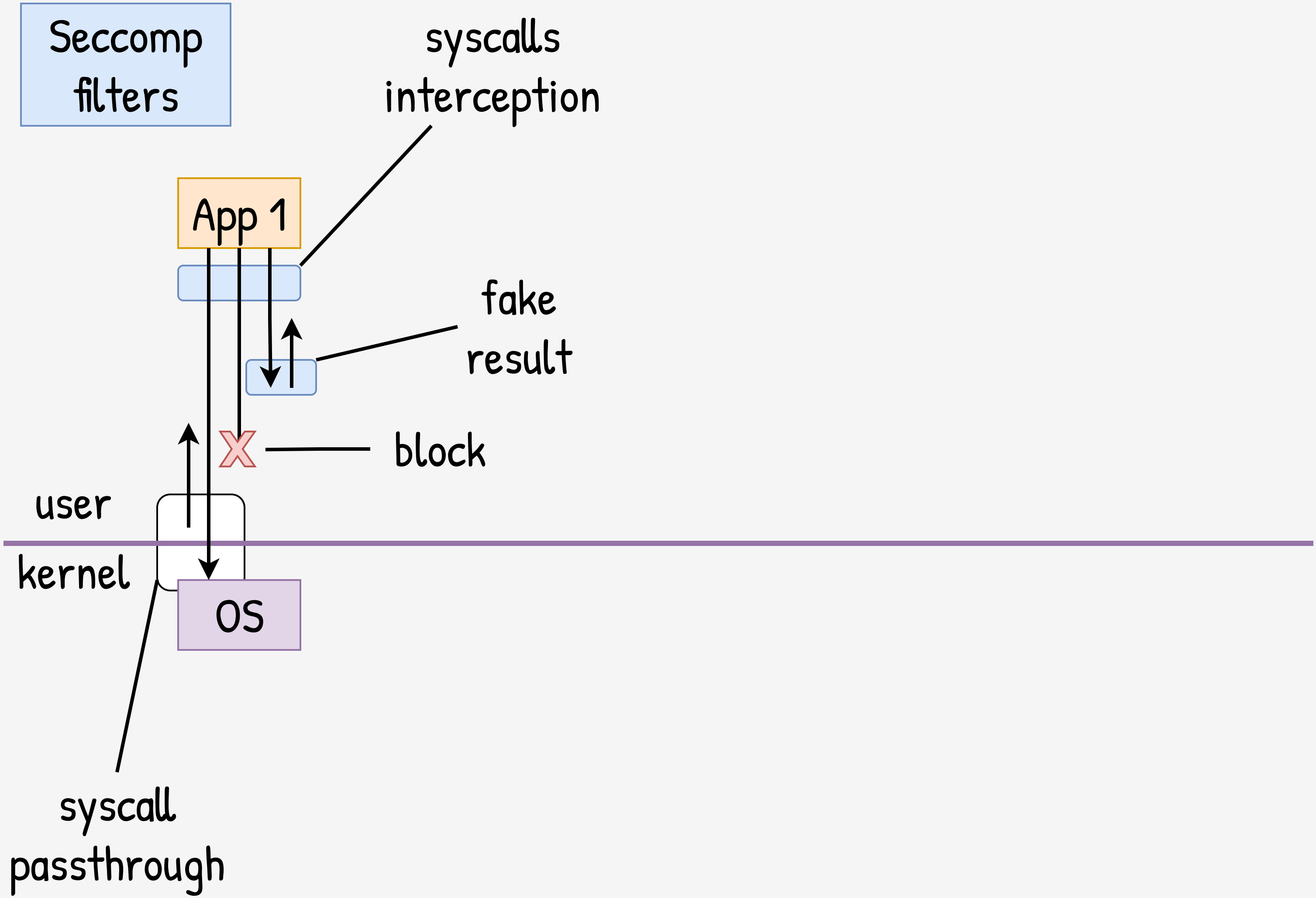

user
ping
Raw Net Cap
Broadcast Cap
resolve DNS
craft ICMP
send 1 dst
broadcast
network
output
parse
recv response
Segregation of root's powers into capabilities

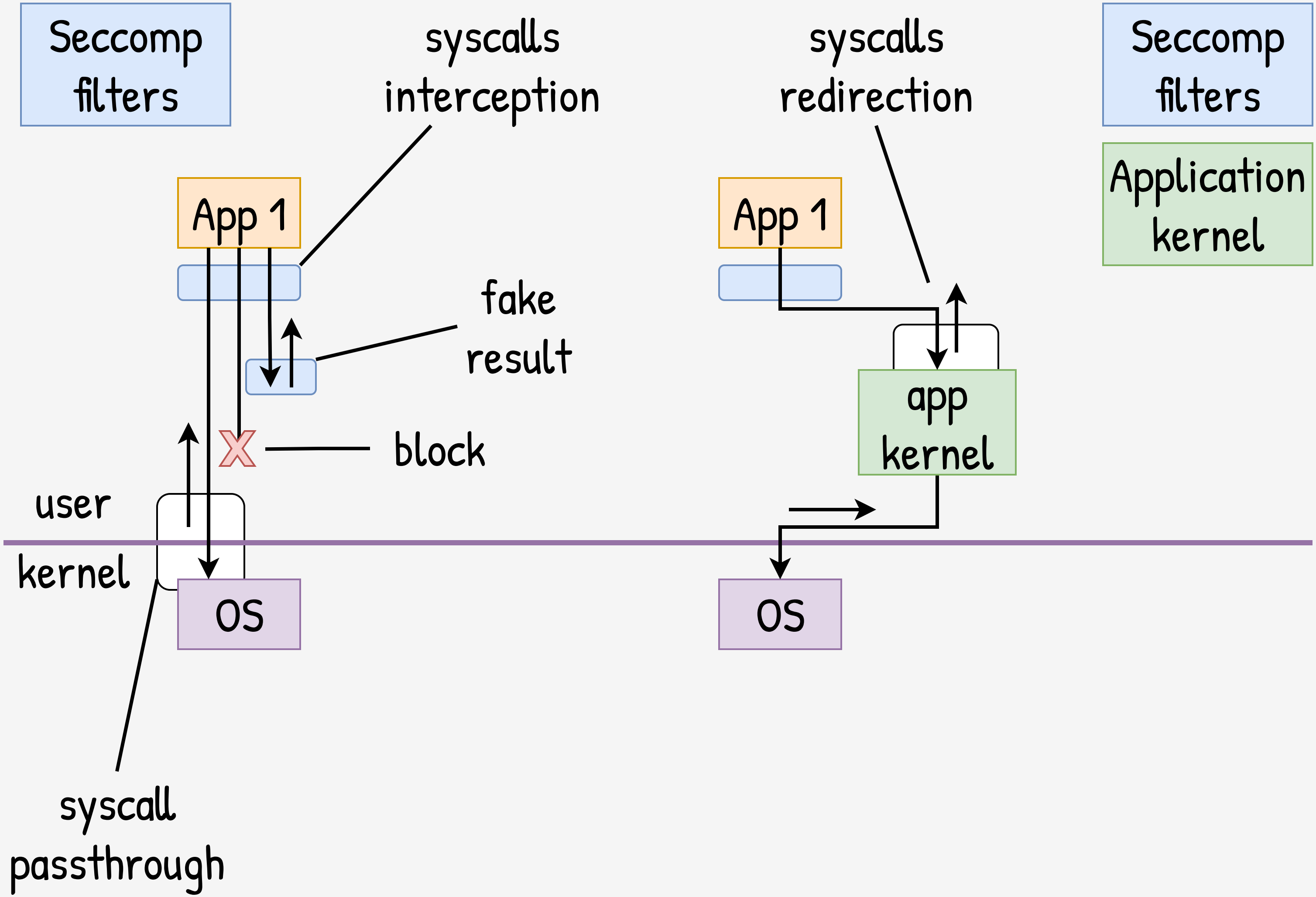

user
ping
Raw Net Cap
Broadcast Cap
Raw Net &
Broadcast Caps
resolve DNS
craft ICMP
send 1 dst
broadcast
network
output
parse
recv response
Segregation of
root's powers
into capabilities
a bug anywhere does
not lead to full root


filter/simulate syscalls
CPU
Mem
App 1
App 2
syscalls
user
kernel
FS
IPC
Net
Users
kernel services

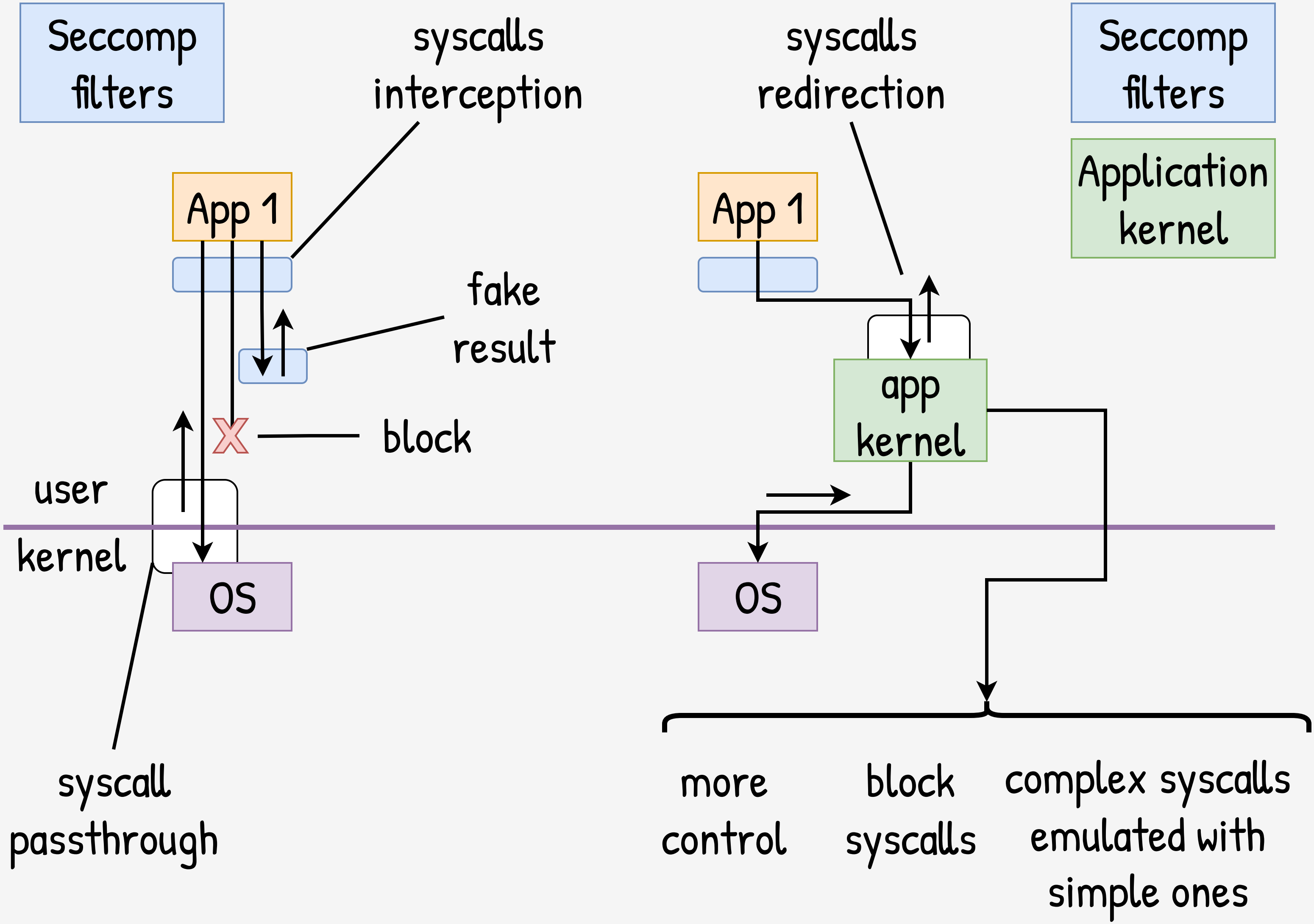

App 1
user
kernel
OS

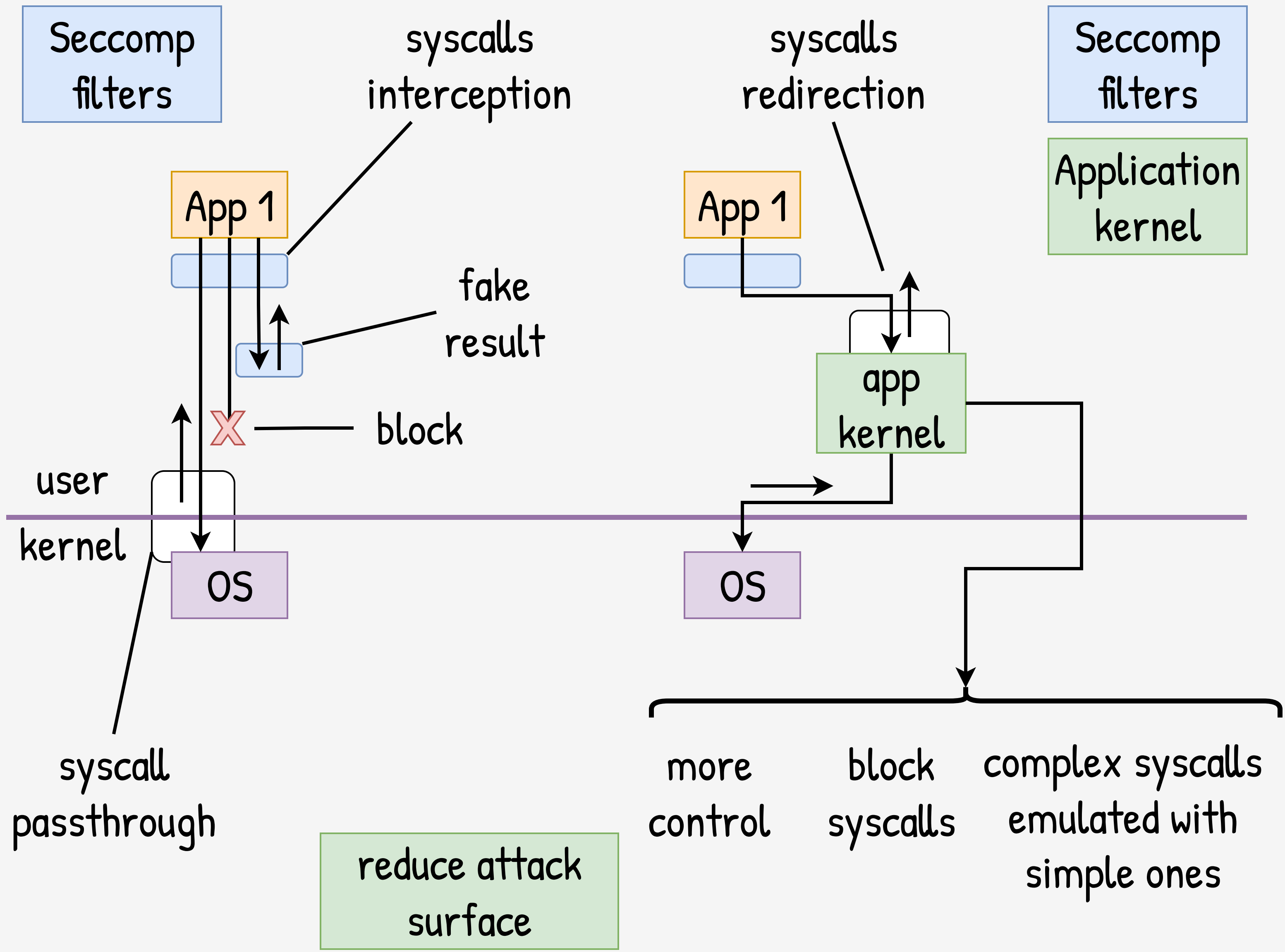

Seccomp filters
syscalls interception
App 1
user
kernel
OS
syscall passthrough


Seccomp filters
syscalls interception
App 1
fake result
block
user
kernel
OS
syscall passthrough

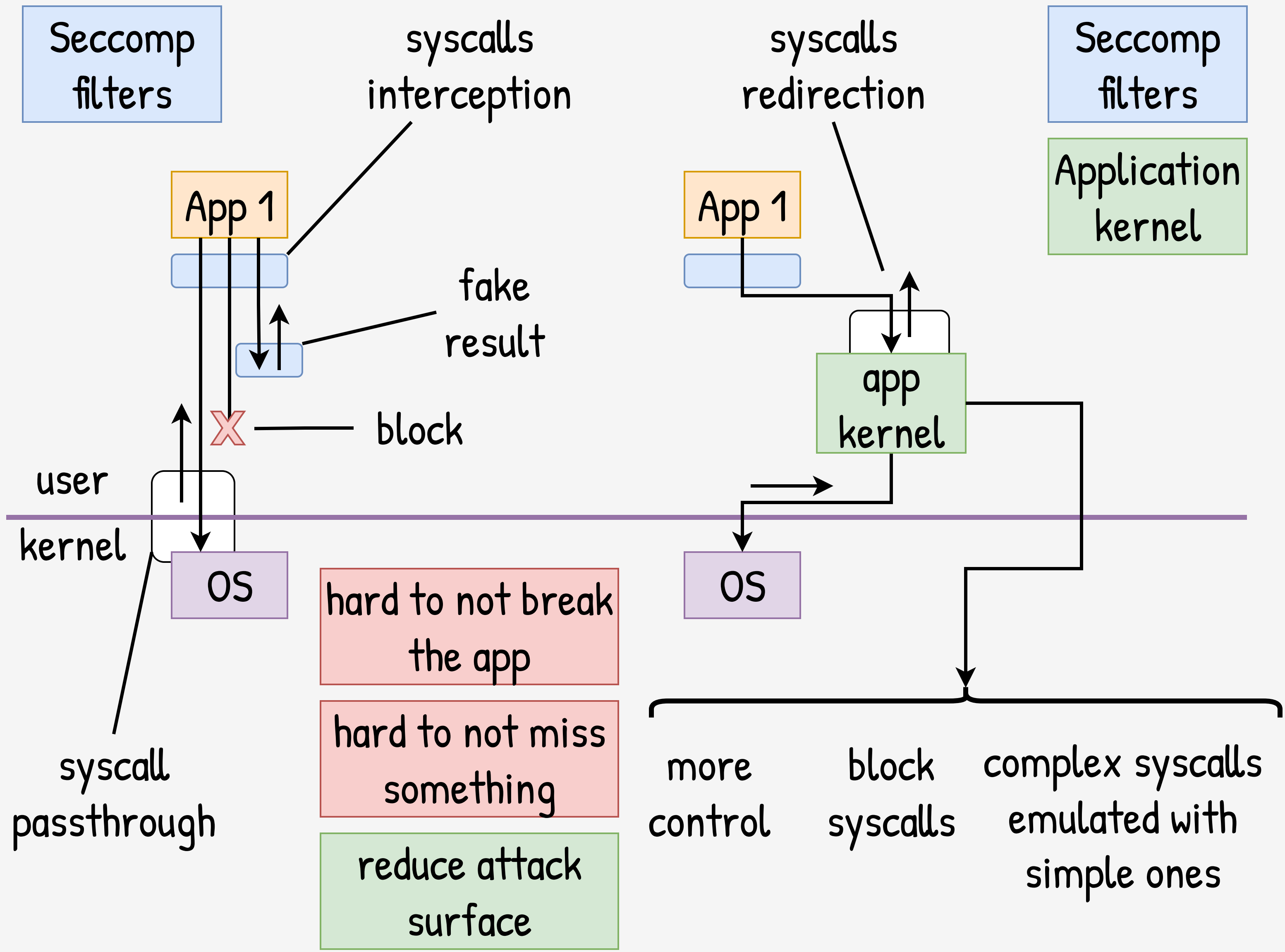


Seccomp filters
syscalls interception
syscalls redirection
Seccomp filters
Application kernel
App 1
App 1
fake result
app kernel
block
user
kernel
OS
OS
syscall passthrough

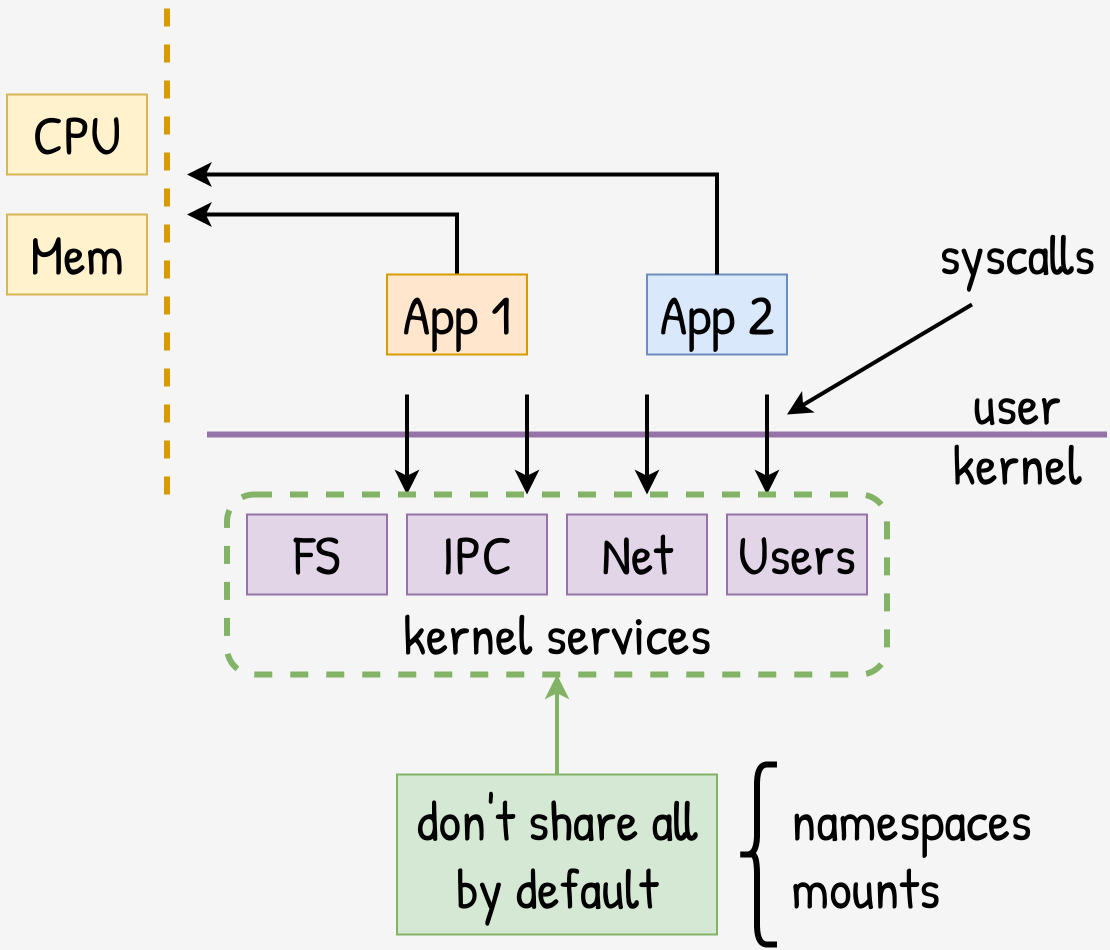

Seccomp filters
syscalls interception
syscalls redirection
Seccomp filters
Application kernel
App 1
App 1
fake result
app kernel
block
user
kernel
OS
OS
syscall passthrough
more control
block syscalls
complex syscalls emulated with simple ones

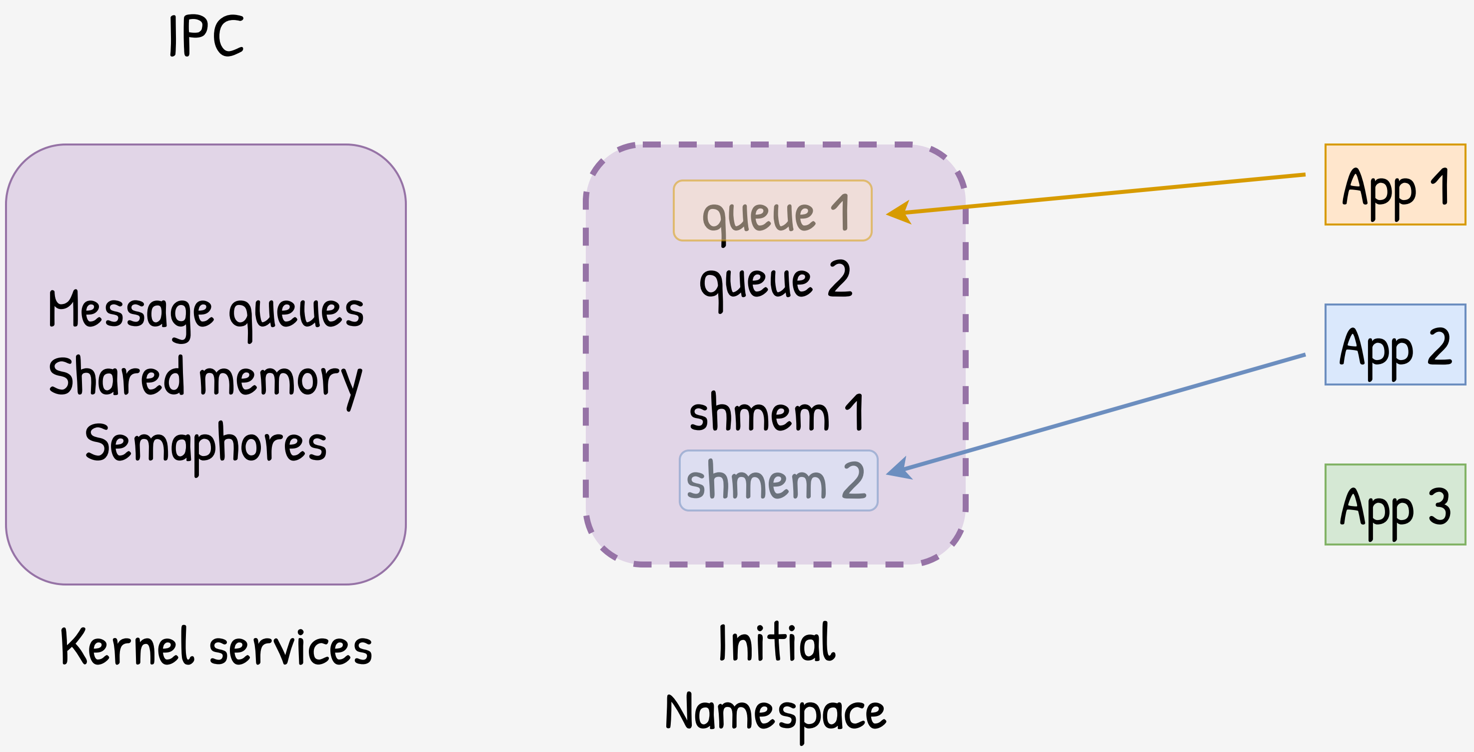


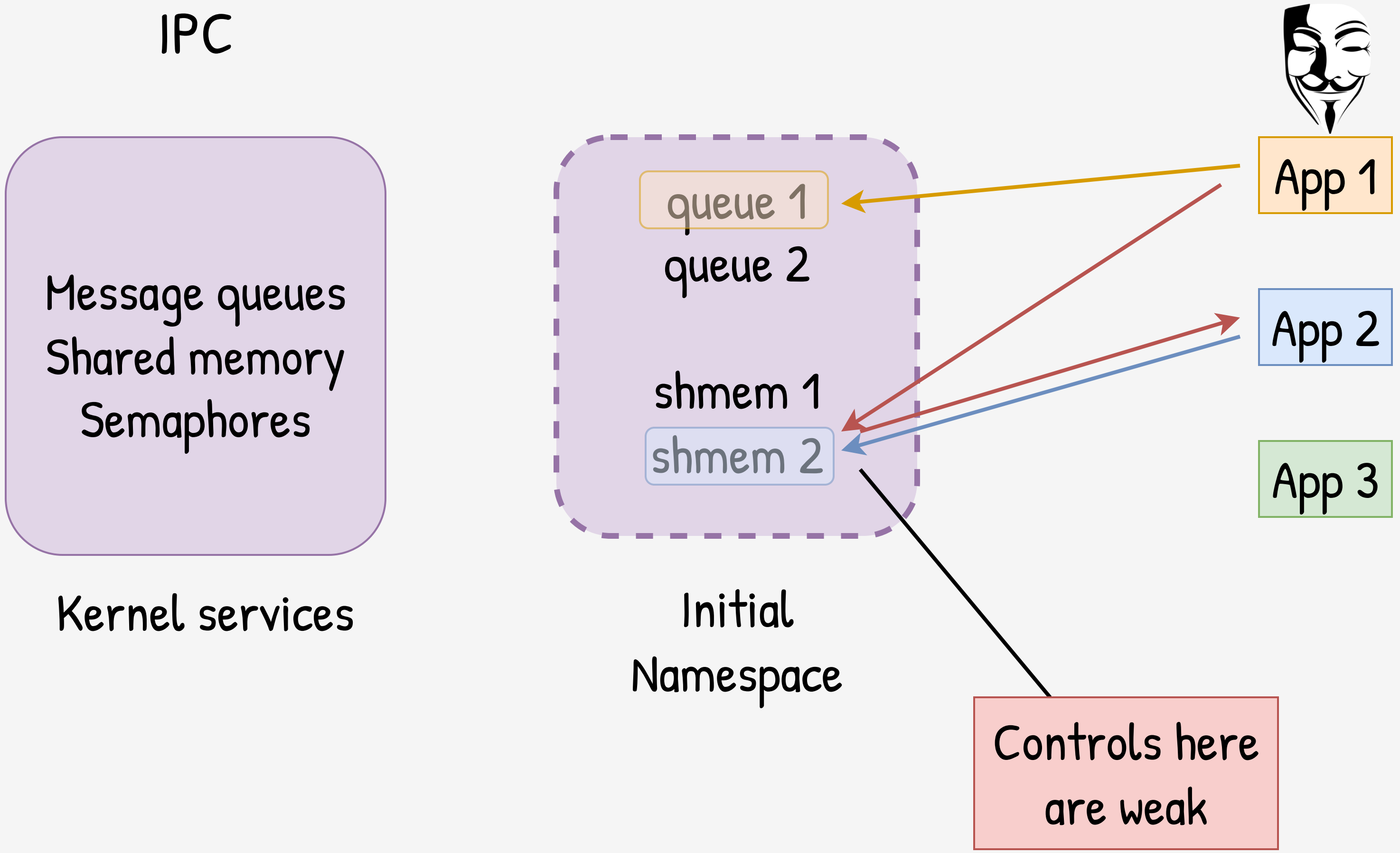
Seccomp filters
syscalls interception
syscalls redirection
Seccomp filters
Application kernel
App 1
App 1
fake result
app kernel
block
user
kernel
OS
OS
syscall passthrough
reduce attack surface
more control
block syscalls
complex syscalls emulated with simple ones



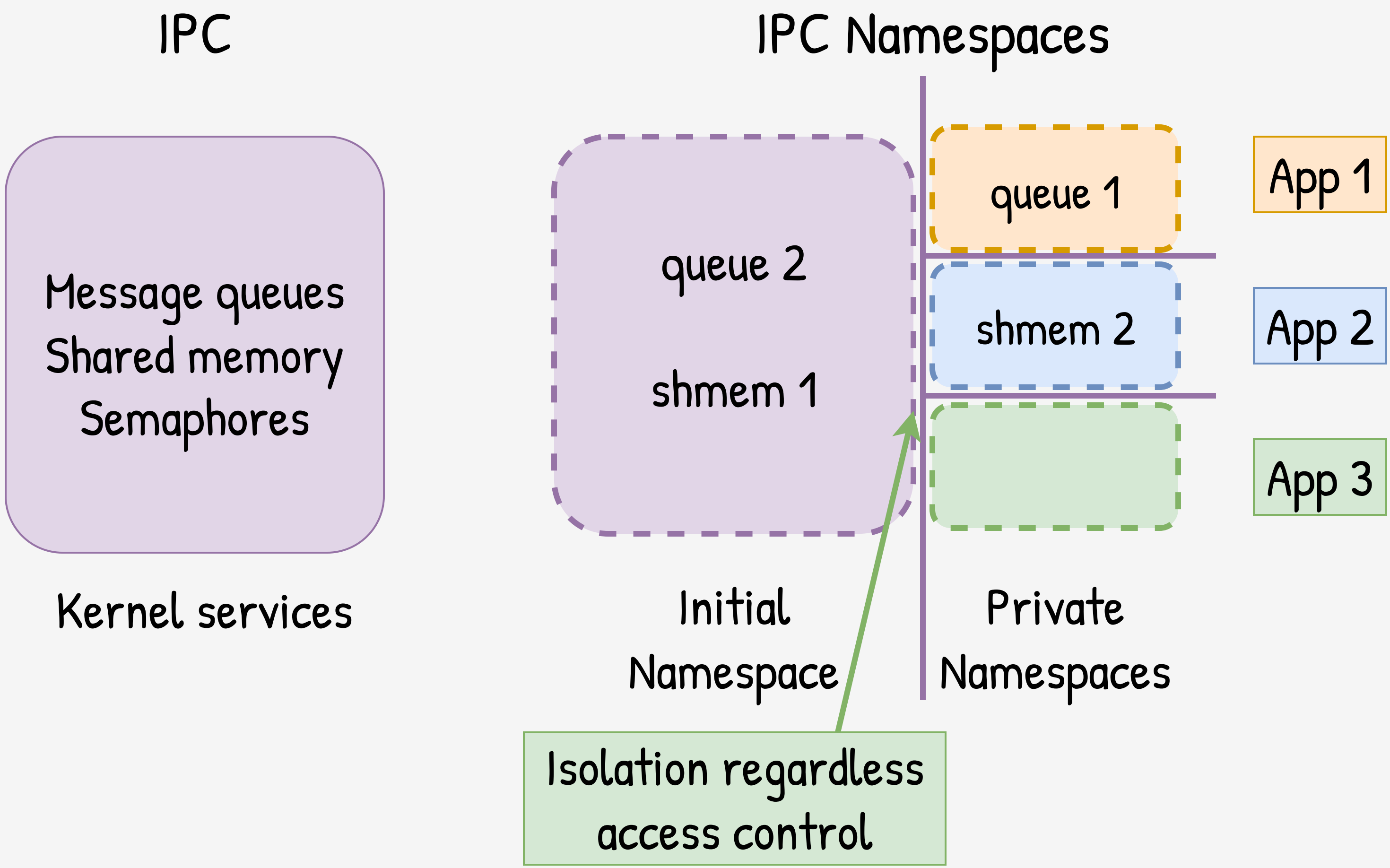
Seccomp filters
syscalls interception
syscalls redirection
Seccomp filters
Application kernel
App 1
App 1
fake result
app kernel
block
user
kernel
OS
OS
hard to not break the app
hard to not miss something
reduce attack surface
syscall passthrough
more control
block syscalls
complex syscalls emulated with simple ones

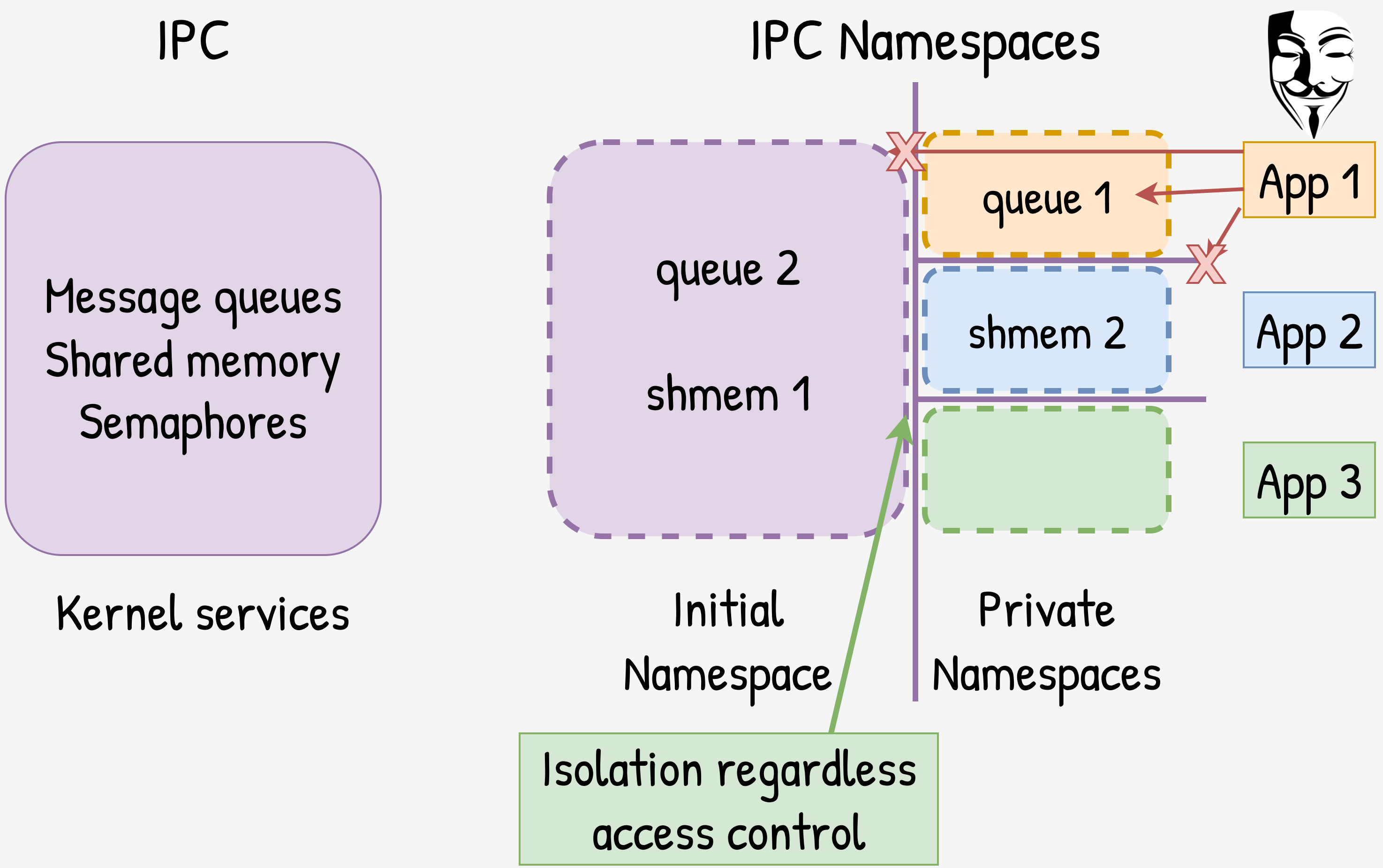

CPU
Mem
App 1
App 2
syscalls
user
kernel
FS
IPC
Net
Users
kernel services
don't share all
by default
namespaces
mounts

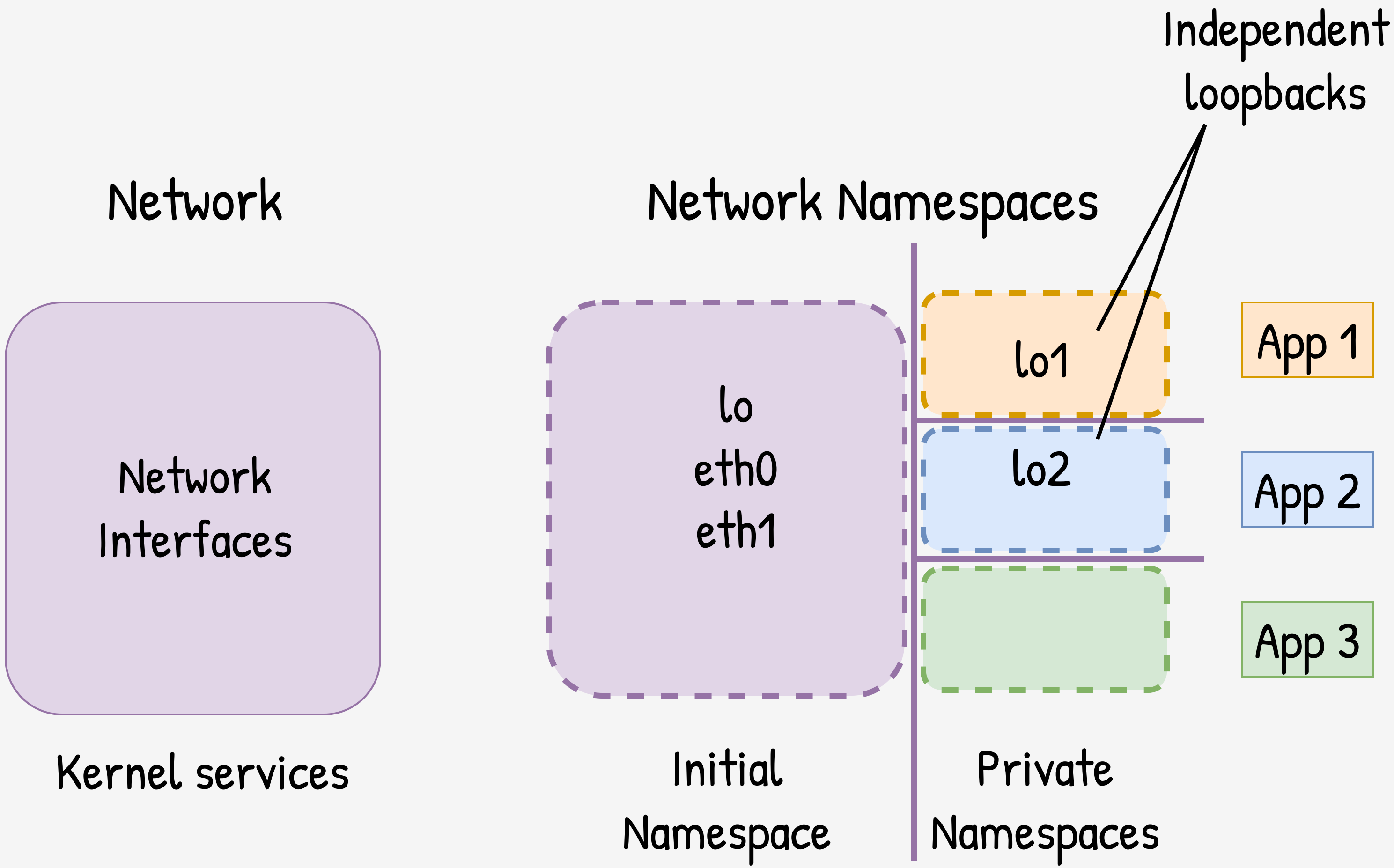

IPC
Message queues
Shared memory
Semaphores
Kernel services
queue 1
queue 2
shmem 1
shmem 2
Initial
Namespace
App 1
App 2
App 3

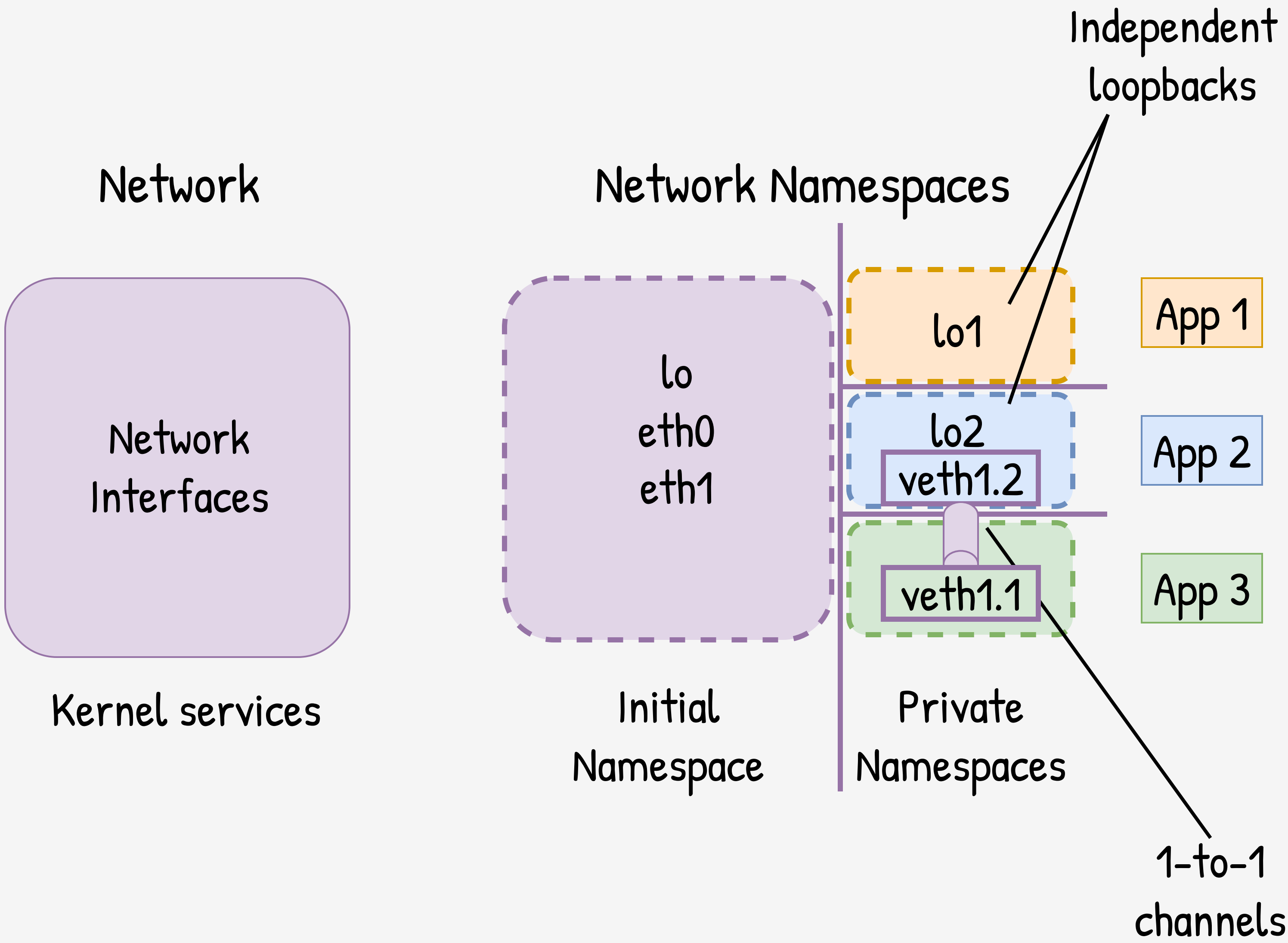


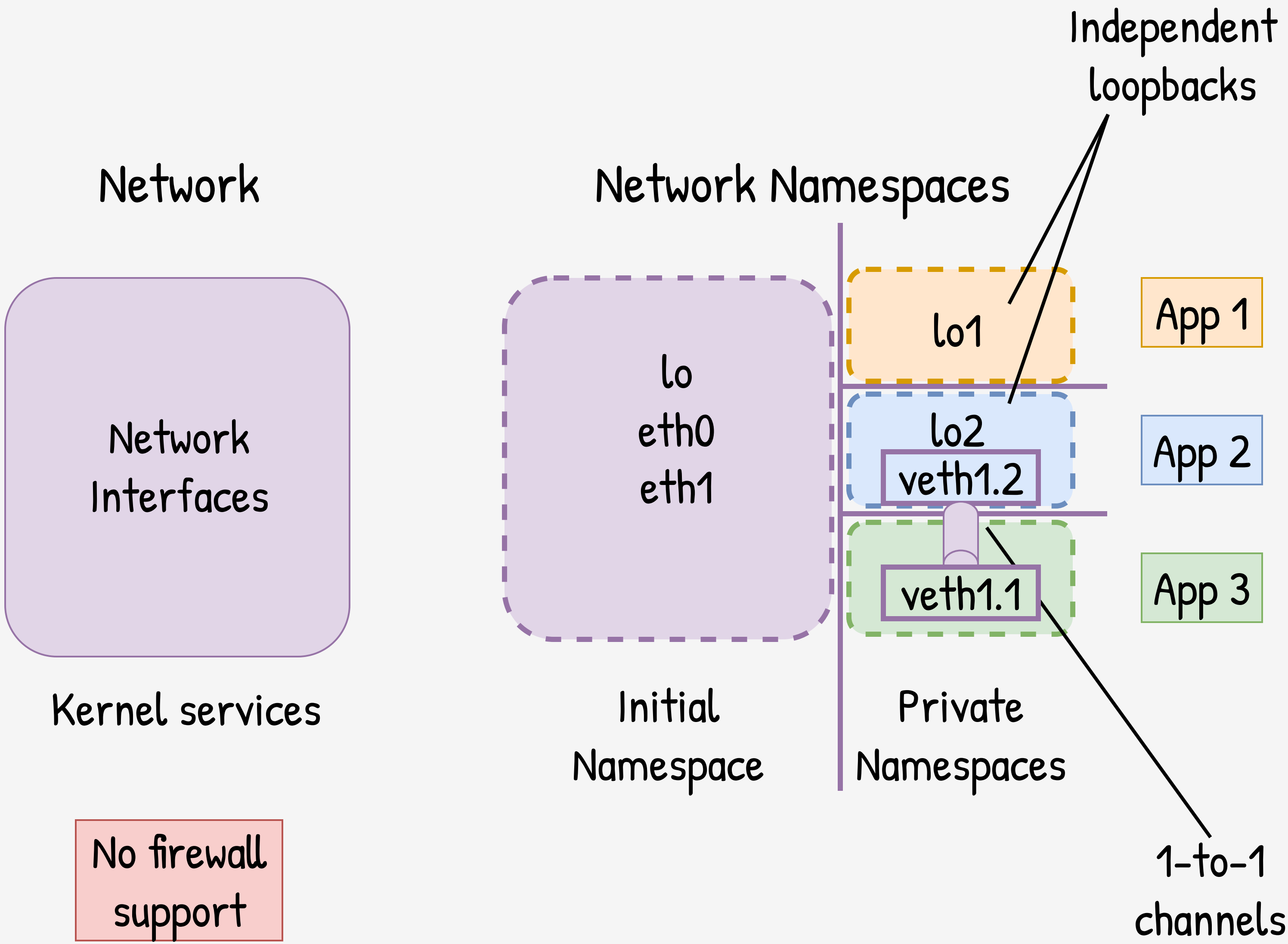
IPC
Message queues
Shared memory
Semaphores
Kernel services
queue 1
queue 2
shmem 1
shmem 2
Initial
Namespace
App 1
App 2
App 3
Controls here
are weak


IPC
Message queues
Shared memory
Semaphores
Kernel services
IPC Namespaces
queue 2
shmem 1
queue 1
shmem 2
App 1
App 2
App 3
Initial Namespace
Private Namespaces
Isolation regardless access control


IPC
IPC Namespaces
Message queues
Shared memory
Semaphores
Kernel services
queue 2
shmem 1
queue 1
shmem 2
App 1
App 2
App 3
Initial
Namespace
Private
Namespaces
Isolation regardless
access control


Independent loopbacks
Network
Network Namespaces
Network Interfaces
lo
eth0
eth1
lo1
lo2
App 1
App 2
App 3
Kernel services
Initial Namespace
Private Namespaces


Network
Network Namespaces
Independent
loopbacks
Network
Interfaces
lo
eth0
eth1
lo1
lo2
veth1.2
veth1.1
App 1
App 2
App 3
Kernel services
Initial
Namespace
Private
Namespaces
1-to-1
channels

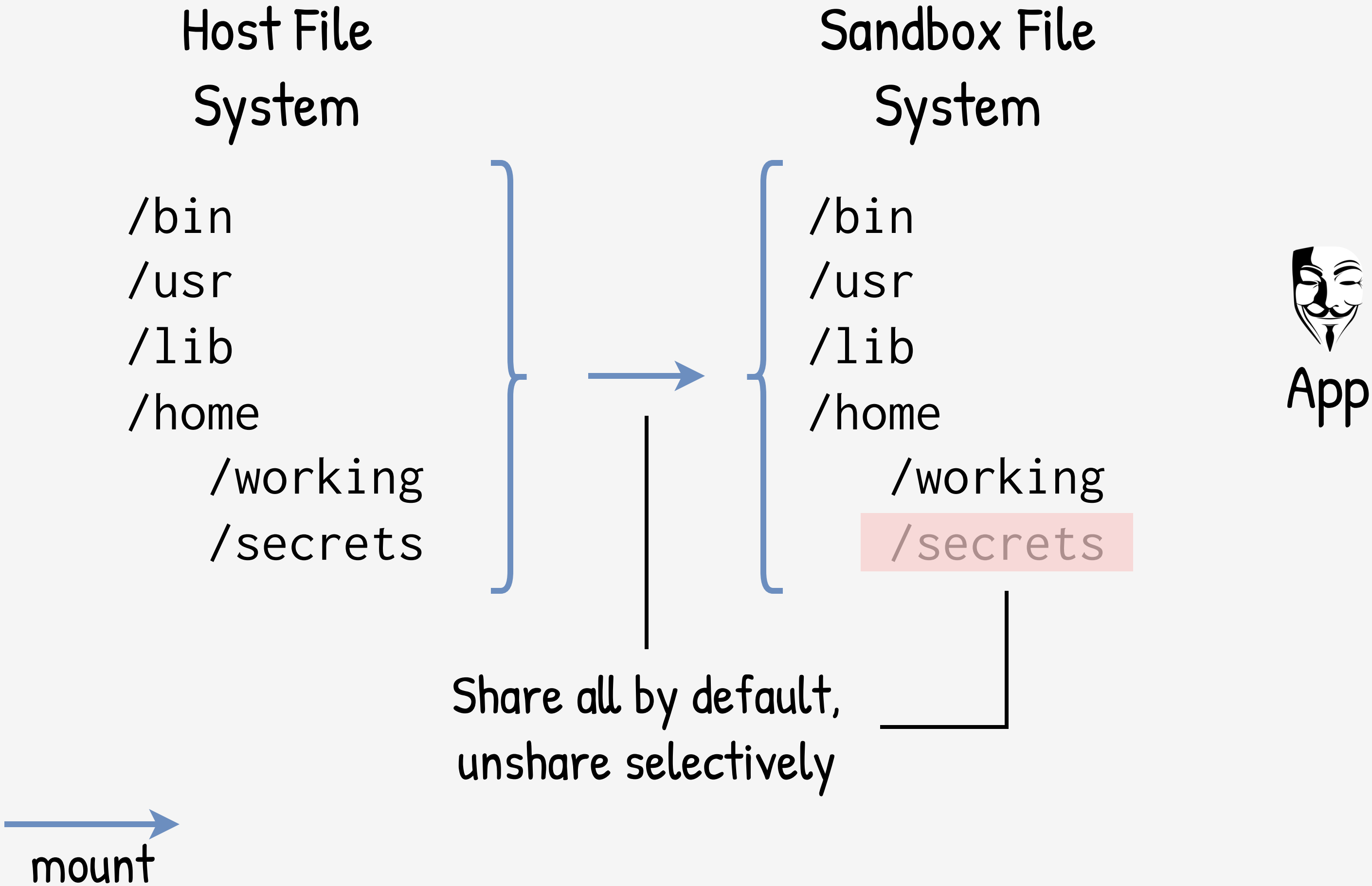

Network
Network Interfaces
Kernel services
No firewall support
Network Namespaces
Independent loopbacks
lo
eth0
eth1
lo1
lo2
veth1.2
veth1.1
App 1
App 2
App 3
Initial Namespace
Private Namespaces
1-to-1 channels

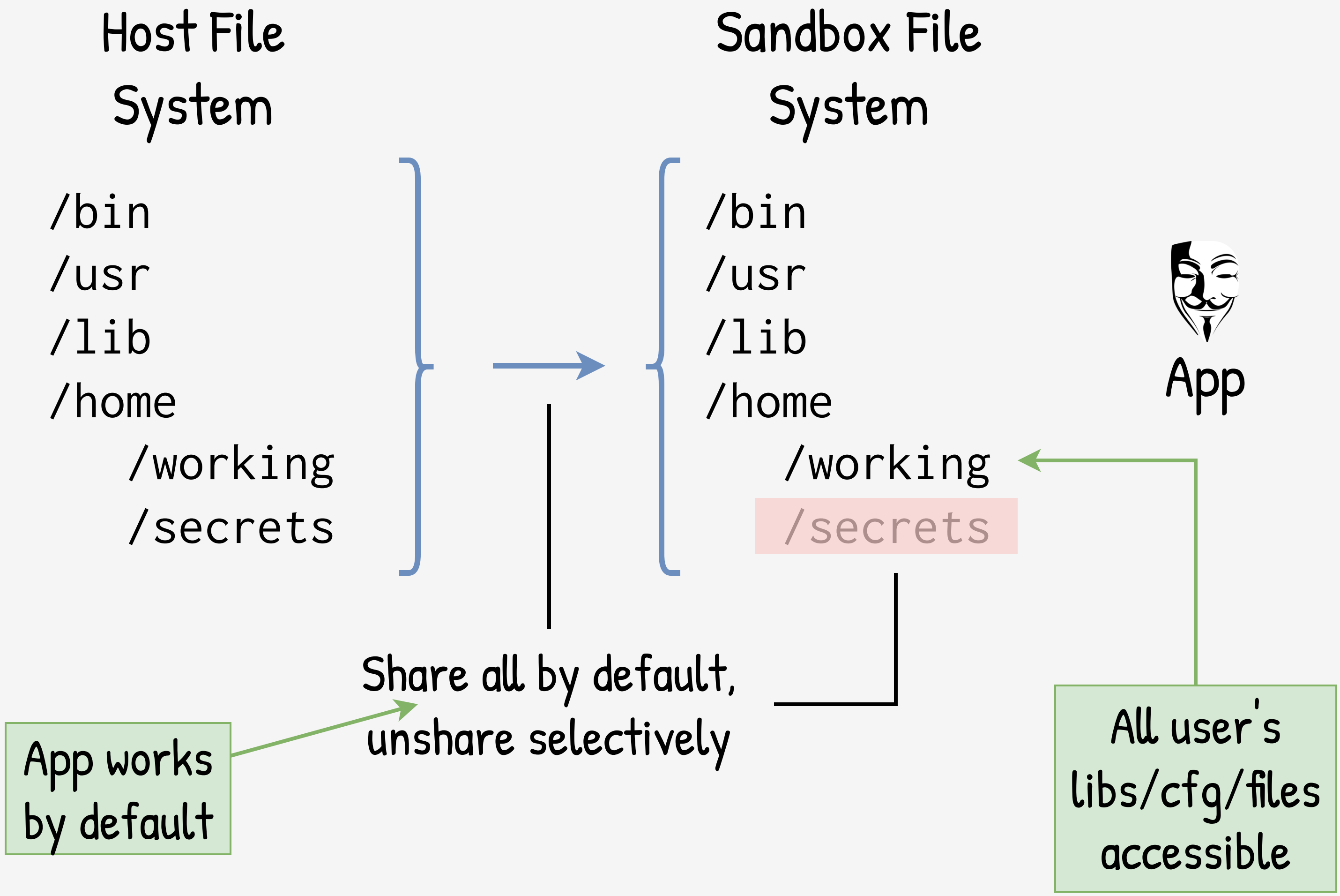
Host File System

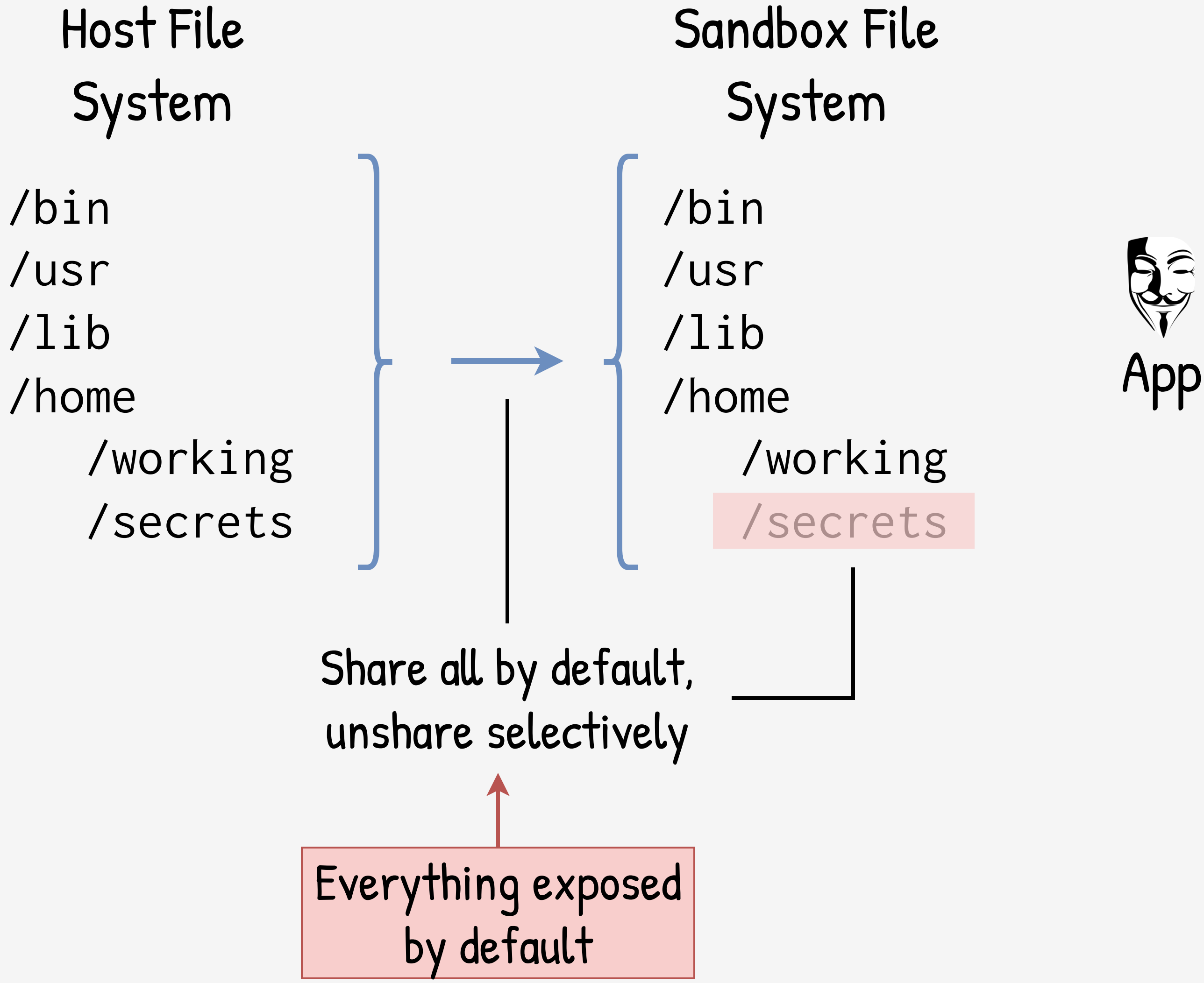
```
/bin
/usr
/lib
/home
    /working
    /secrets
```

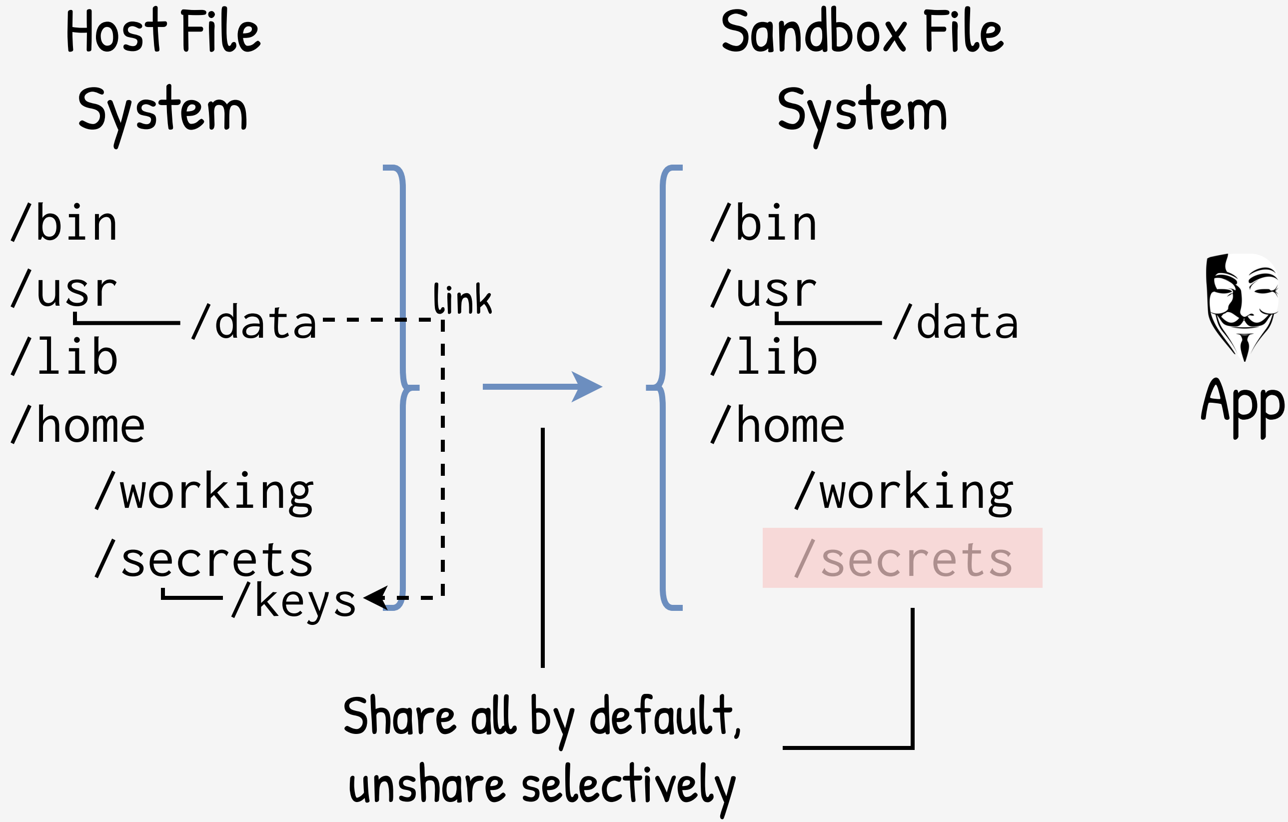
Sandbox File System

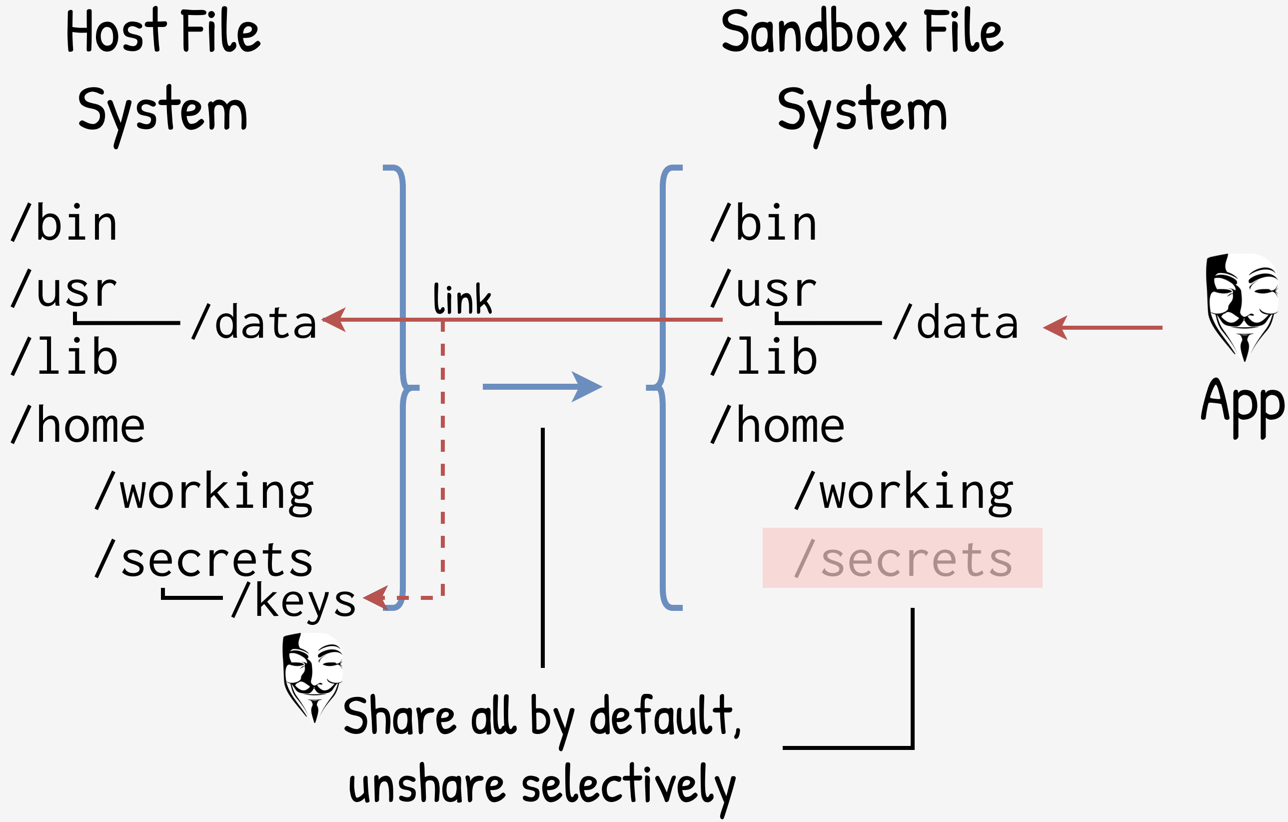
App

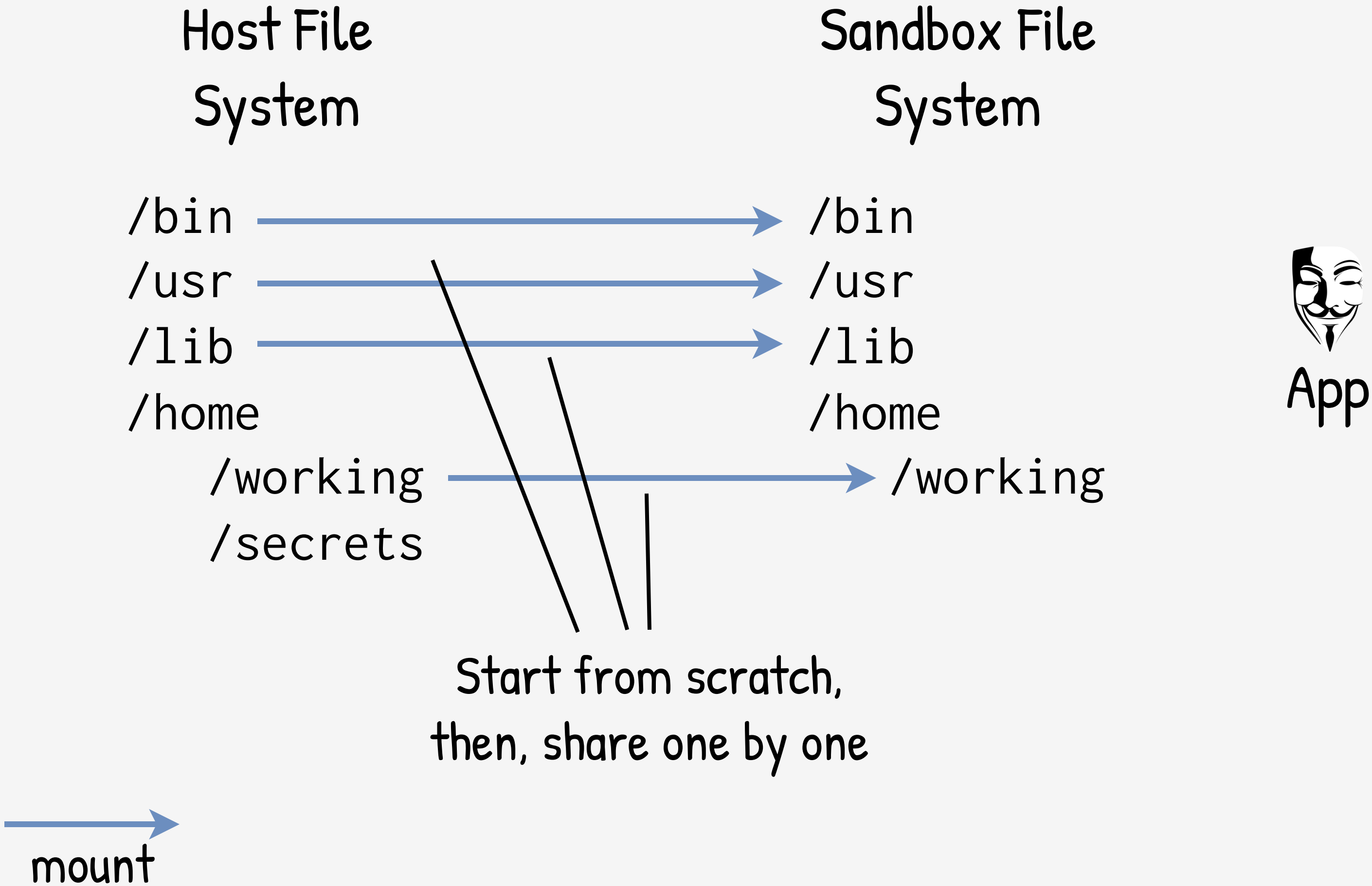

Host File System
/bin
/usr
/lib
/home
/working
/secrets
Sandbox File System
/bin
/usr
/lib
/home
/working
/secrets
App
Share all by default, unshare selectively
mount

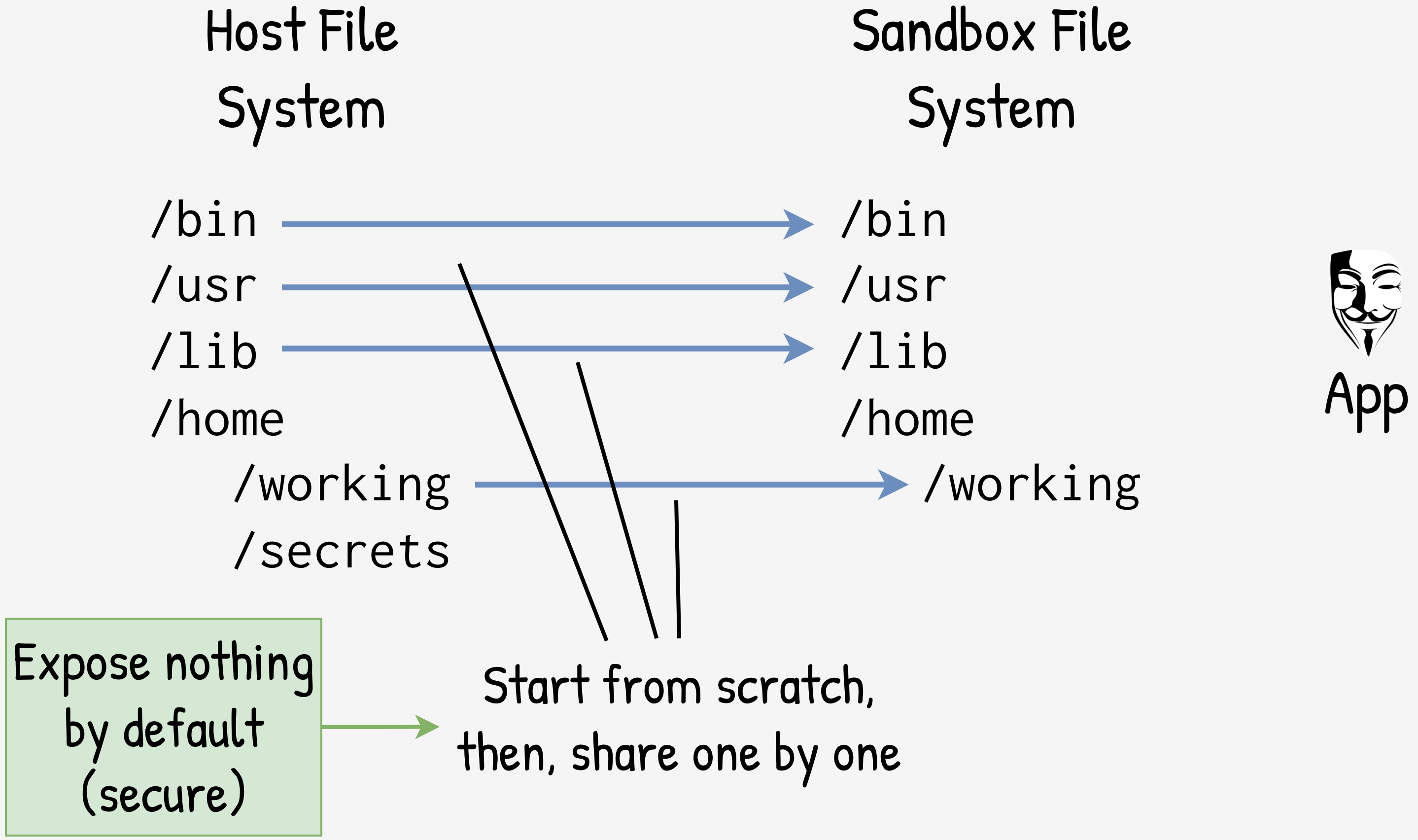

Host File System
/bin
/usr
/lib
/home
/working
/secrets
Sandbox File System
/bin
/usr
/lib
/home
/working
/secrets
App
Share all by default,
unshare selectively
App works by default
All user's libs/cfg/files accessible

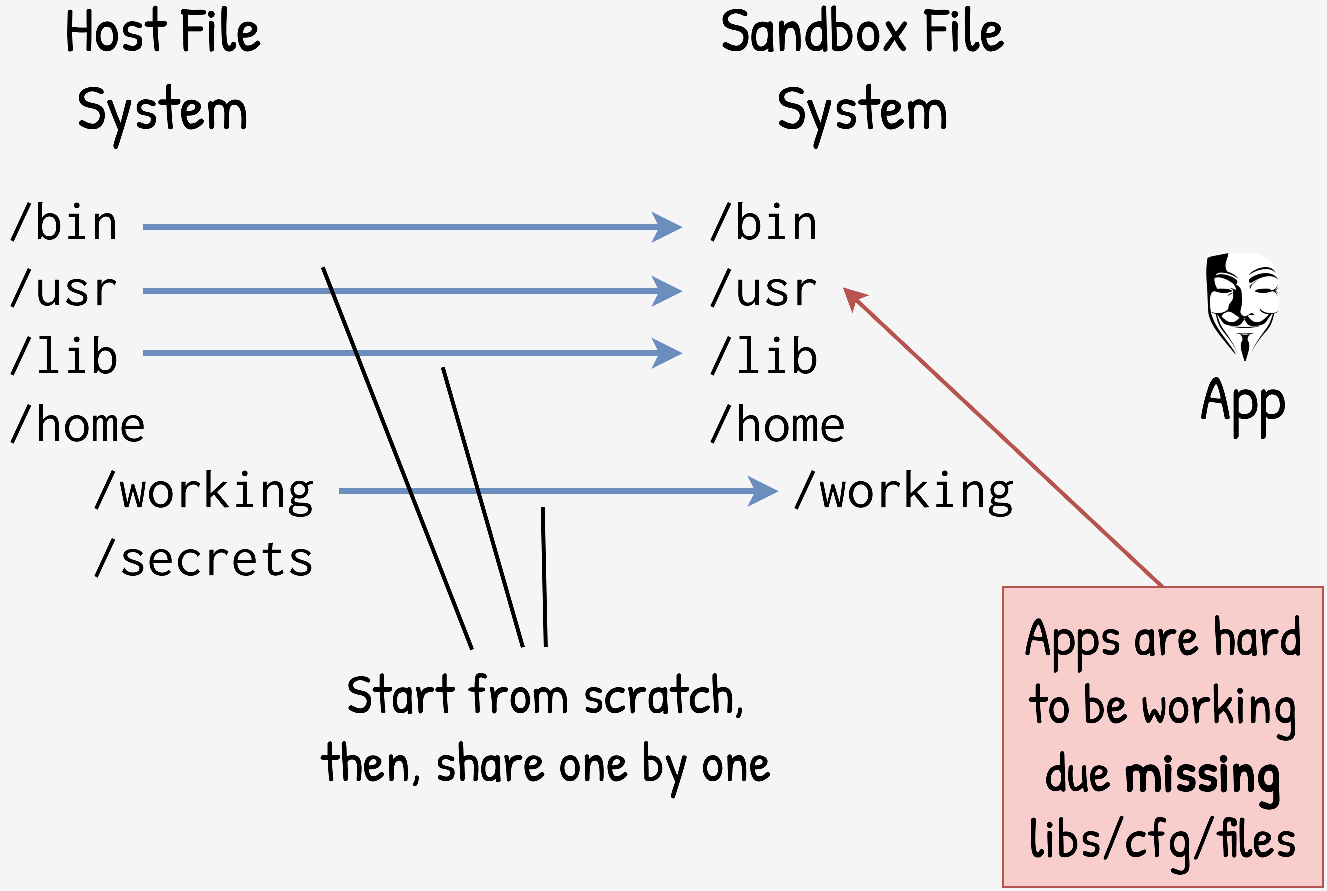

Host File System
/bin
/usr
/lib
/home
/working
/secrets
Sandbox File System
/bin
/usr
/lib
/home
/working
/secrets
App
Share all by default, unshare selectively
Everything exposed by default

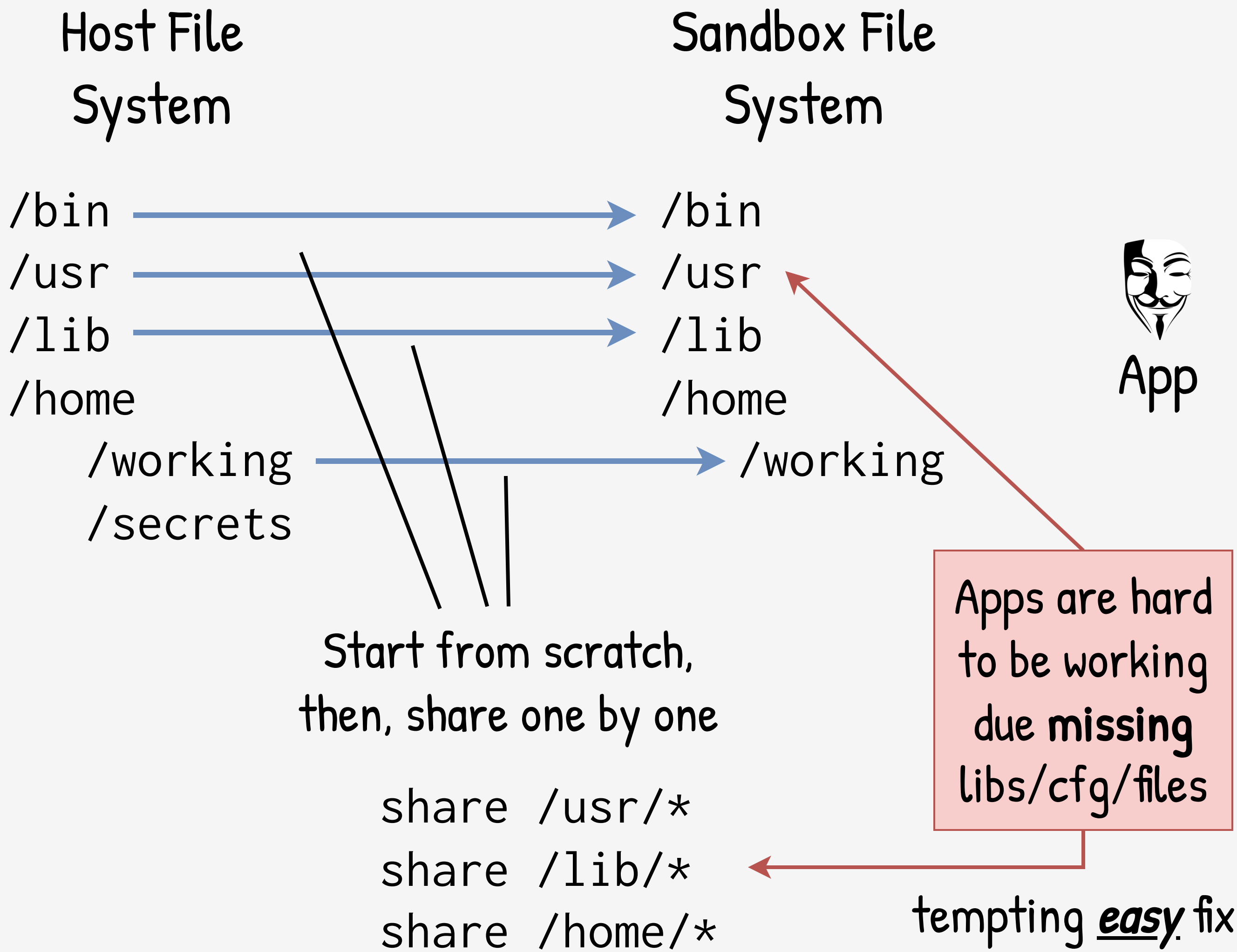

Host File System
/bin
/usr
/data
/lib
/home
/working
/secrets
/keys
link
Sandbox File System
/bin
/usr
/data
/lib
/home
/working
/secrets
App
Share all by default,
unshare selectively

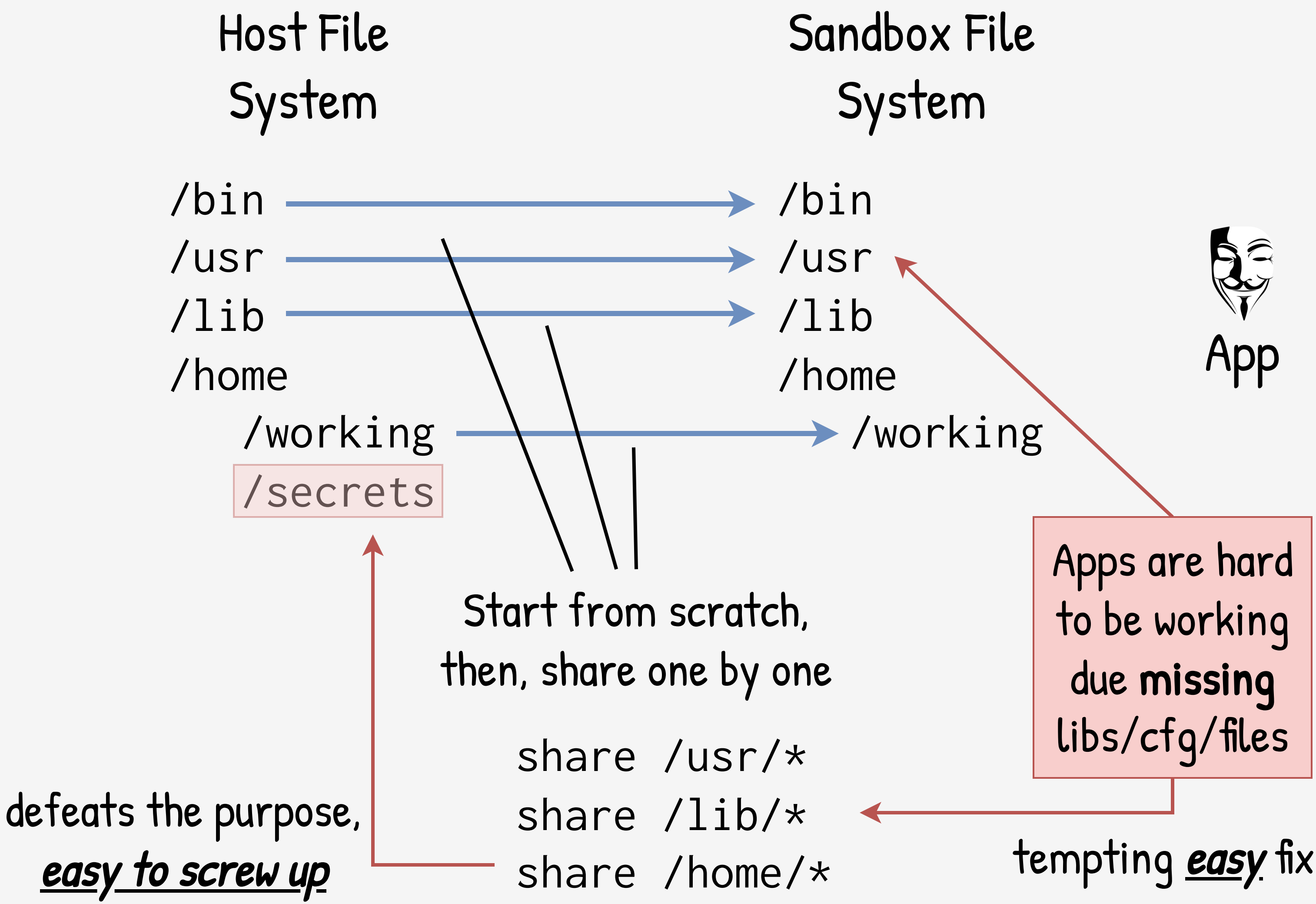

Host File System
Sandbox File System
/bin
/usr
/data
/lib
/home
/working
/secrets
/keys
link
/bin
/usr
/data
/lib
/home
/working
/secrets
App
Share all by default,
unshare selectively

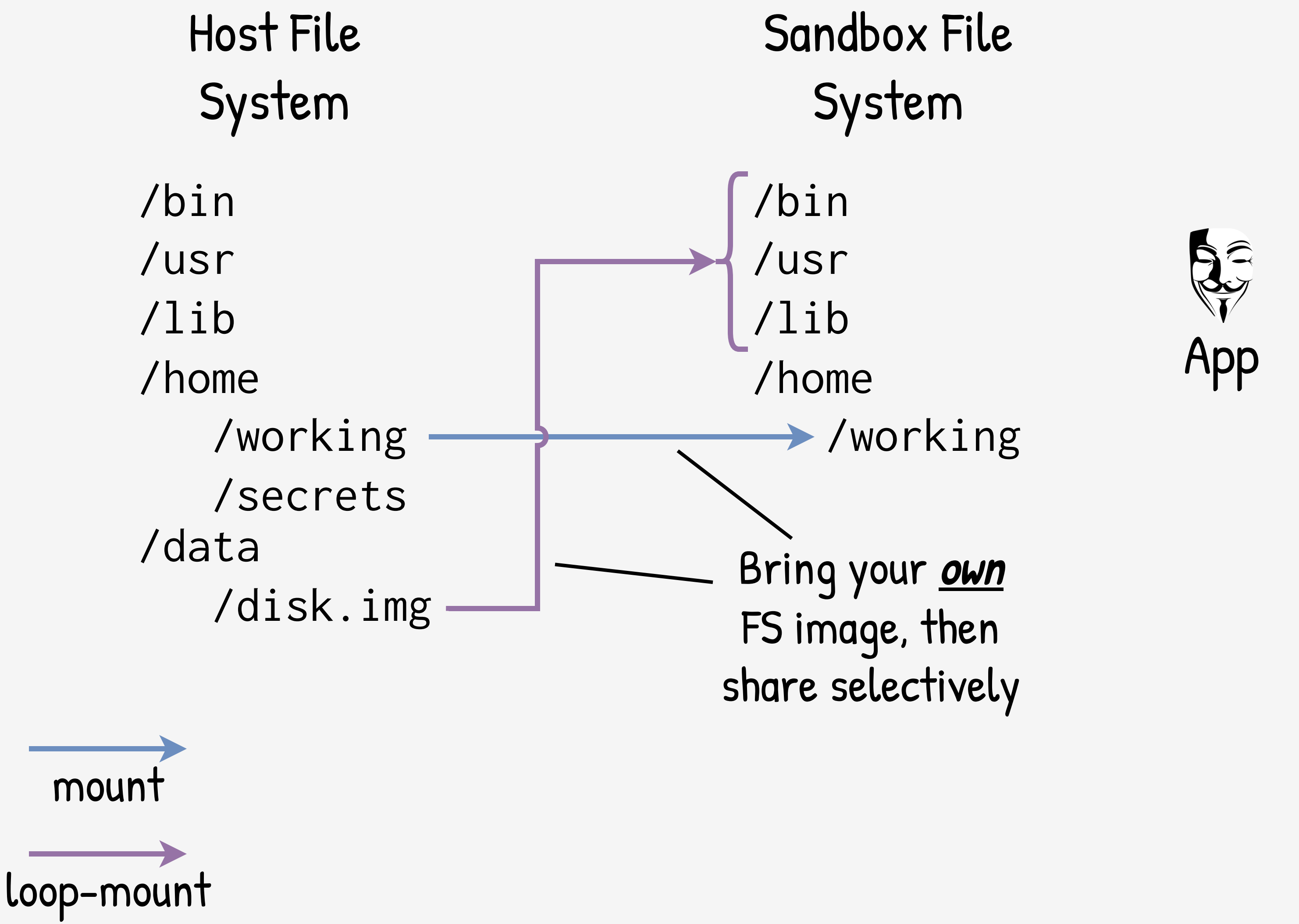

Host File System
Sandbox File System
/bin
/usr
/lib
/home
/working
/secrets
/bin
/usr
/lib
/home
/working
App
Start from scratch, then, share one by one
mount

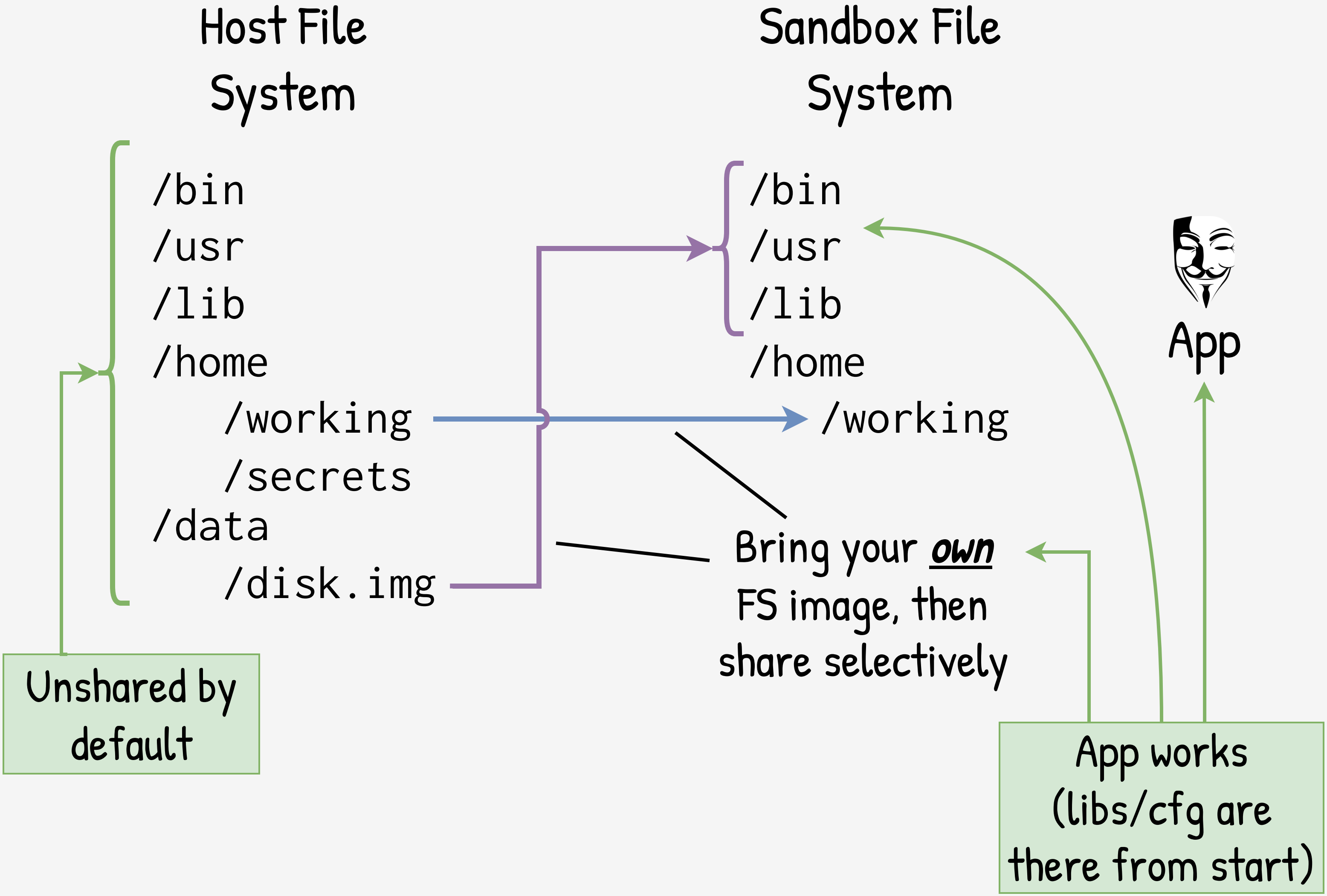

Host File System
Sandbox File System
/bin
/usr
/lib
/home
/working
/secrets
/bin
/usr
/lib
/home
/working
App
Expose nothing by default (secure)
Start from scratch, then, share one by one

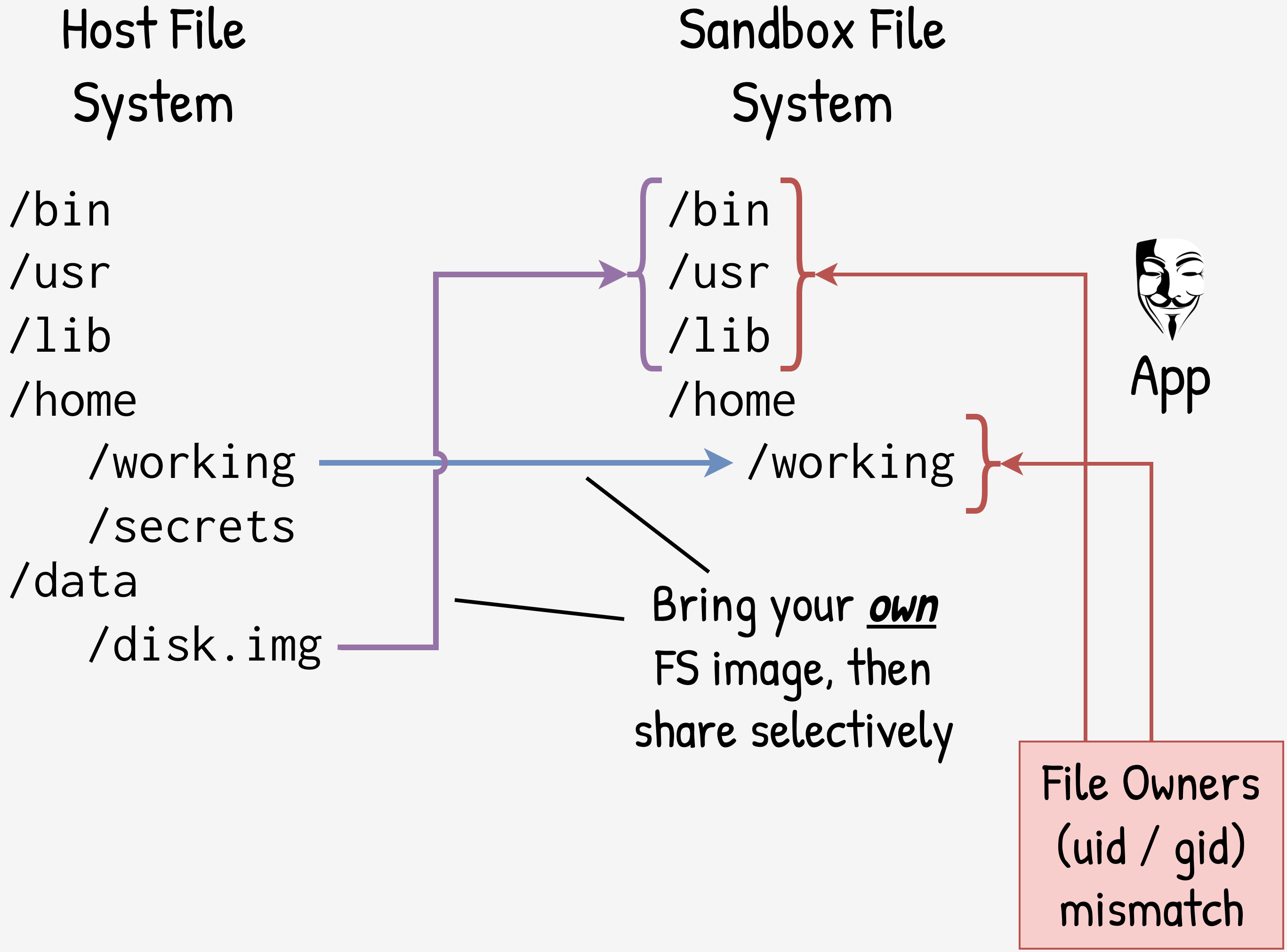

Host File System
Sandbox File System
/bin
/usr
/lib
/home
/working
/secrets
/bin
/usr
/lib
/home
/working
App
Start from scratch, then, share one by one
Apps are hard to be working due **missing** libs/cfg/files

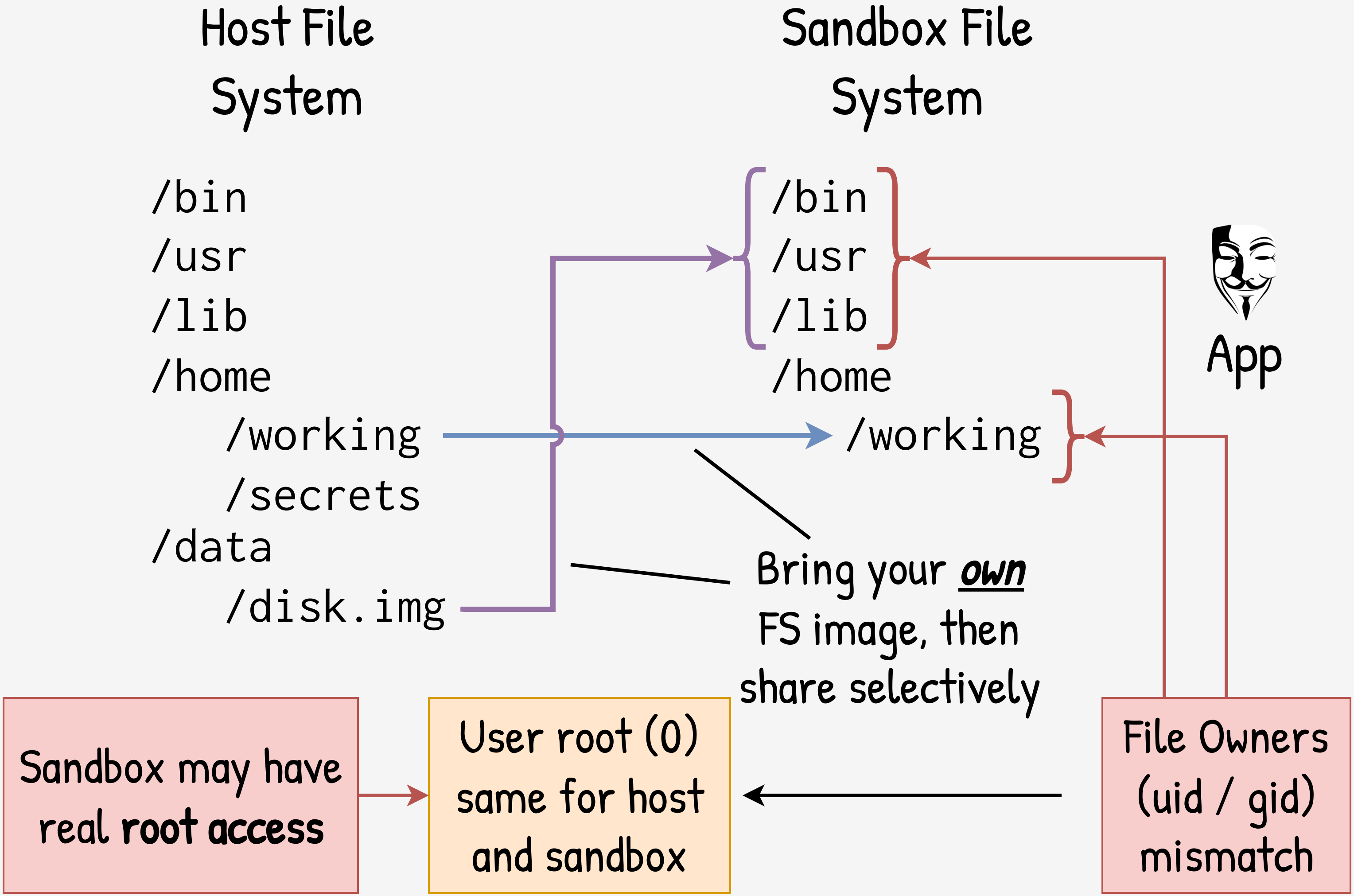

Host File System
Sandbox File System
/bin
/usr
/lib
/home
/working
/secrets
/bin
/usr
/lib
/home
/working
App
Start from scratch, then, share one by one
share /usr/*
share /lib/*
share /home/*
Apps are hard to be working due **missing** libs/cfg/files
tempting ***easy*** fix

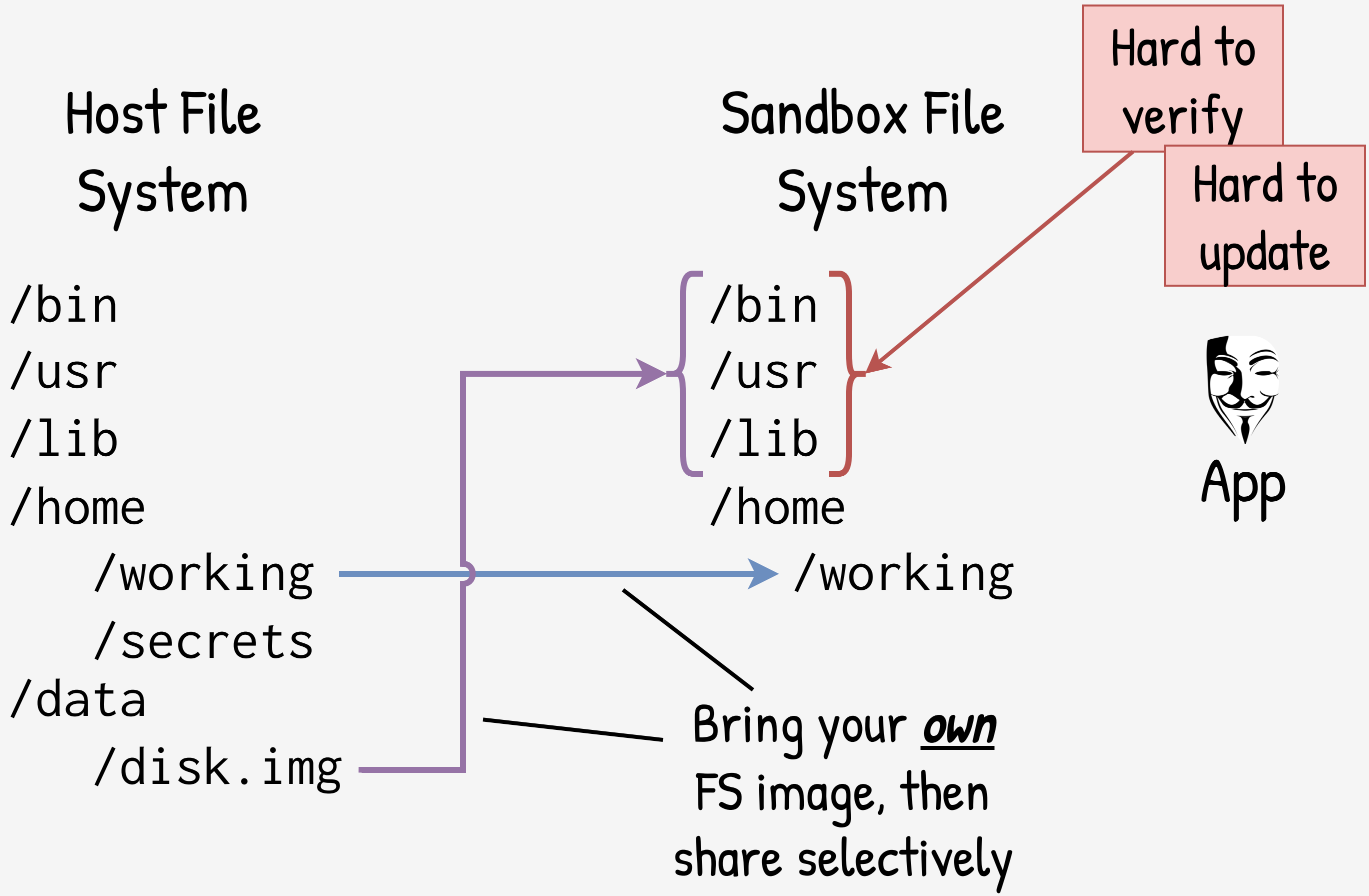

Host File System
Sandbox File System
/bin
/usr
/lib
/home
/working
/secrets
/bin
/usr
/lib
/home
/working
App
Start from scratch, then, share one by one
share /usr/*
share /lib/*
share /home/*
Apps are hard to be working due **missing** libs/cfg/files
tempting ***easy*** fix
defeats the purpose, ***easy to screw up***


Host File System
/bin
/usr
/lib
/home
/working
/secrets
/data
/disk.img
Sandbox File System
/bin
/usr
/lib
/home
/working
App
Bring your ***own*** FS image, then share selectively
mount
loop-mount

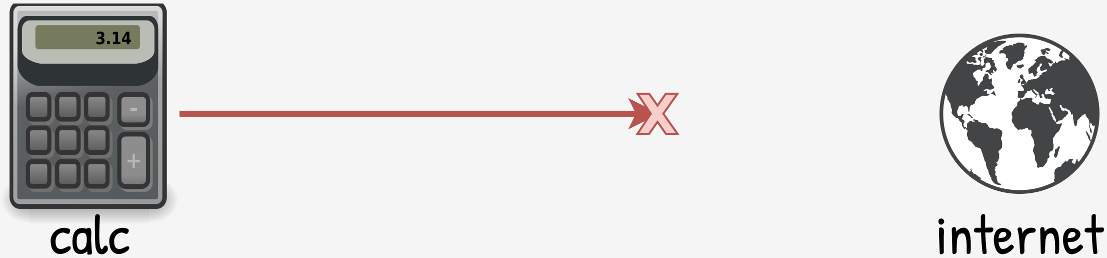

Host File System
/bin
/usr
/lib
/home
/working
/secrets
/data
/disk.img
Sandbox File System
/bin
/usr
/lib
/home
/working
App
Bring your own FS image, then share selectively
Unshared by default
App works (libs/cfg are there from start)


Host File System
/bin
/usr
/lib
/home
/working
/secrets
/data
/disk.img
Sandbox File System
/bin
/usr
/lib
/home
/working
App
Bring your *own* FS image, then share selectively
File Owners (uid / gid) mismatch


Host File System
/bin
/usr
/lib
/home
/working
/secrets
/data
/disk.img
Sandbox File System
/bin
/usr
/lib
/home
/working
App
Bring your own FS image, then share selectively
Sandbox may have real root access
User root (0) same for host and sandbox
File Owners (uid / gid) mismatch


Host File System
/bin
/usr
/lib
/home
/working
/secrets
/data
/disk.img
Sandbox File System
/bin
/usr
/lib
/home
/working
Hard to verify
Hard to update
App
Bring your ***own*** FS image, then share selectively

Sandbox Tech	FS	FS Provider	Sbox Cfg
firejail	shared	you	you/3rd
firejail + appimage	img	you/3rd	you/3rd
bubblewrap	unshared	you	you
flatpak	img	3rd	3rd
snap	img	3rd	3rd
docker	img	you/3rd	you


3.14
calc
internet

Binary-oriented

block socket **syscall**

drop net **capabilities**

apparmor profile block net

Binary-oriented

block socket **syscall**

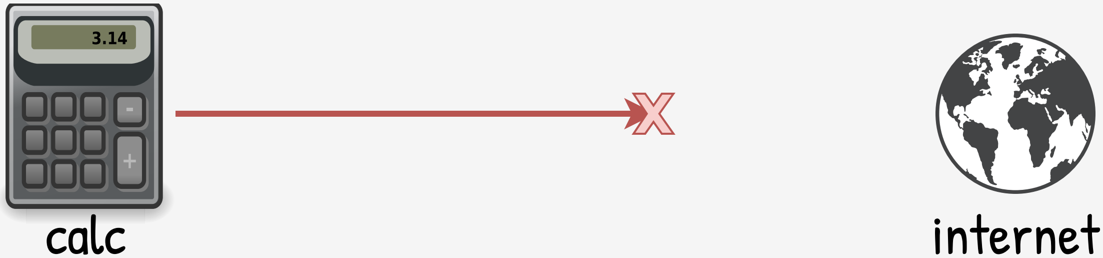
drop net **capabilities**

apparmor profile block net

put process in **namespace** without iface

tag process with **cgroup**; filter with specific **fw** rule

Process-oriented



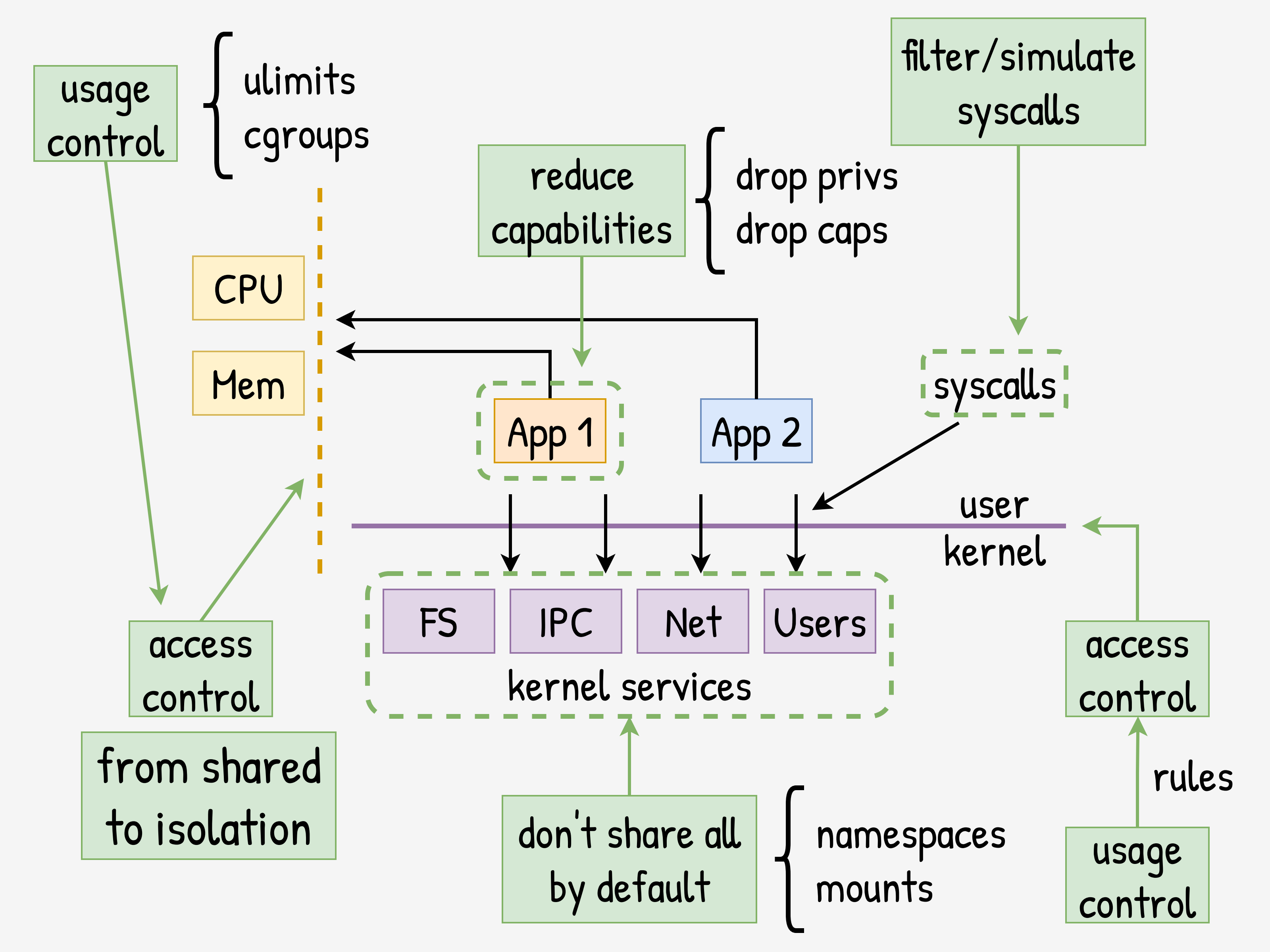
Overlap → Fallback

Binary-oriented

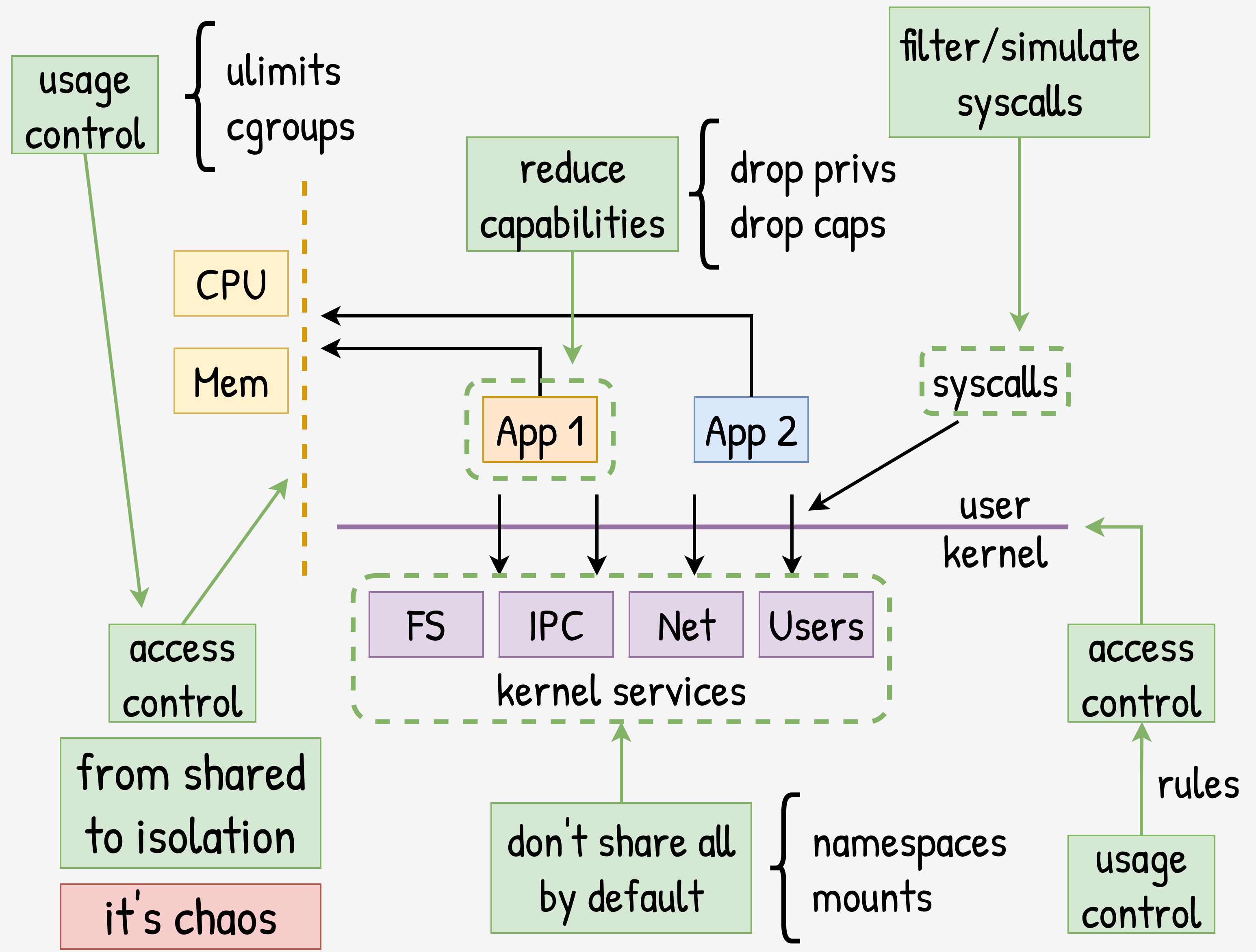
- block socket **syscall**
- drop net **capabilities**
- **apparmor** profile block net

Process-oriented

- put process in **namespace** without iface
- tag process with **cgroup**; filter with specific **fw** rule



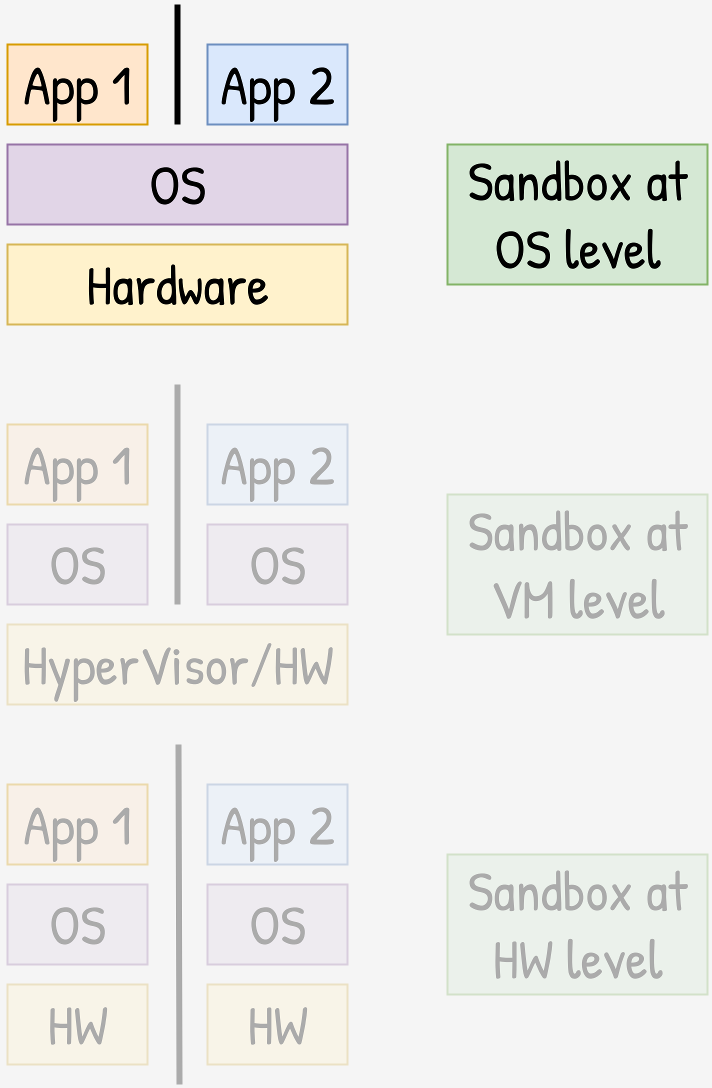
usage control
ulimits
cgroups
reduce capabilities
drop privs
drop caps
filter/simulate syscalls
CPU
Mem
App 1
App 2
syscalls
user
kernel
FS
IPC
Net
Users
kernel services
access control
from shared to isolation
don't share all by default
namespaces
mounts
access control
rules
usage control



usage control
ulimits
cgroups
filter/simulate syscalls
reduce capabilities
drop privs
drop caps
CPU
Mem
App 1
App 2
syscalls
user
kernel
FS
IPC
Net
Users
kernel services
access control
access control
from shared to isolation
it's chaos
don't share all by default
namespaces
mounts
rules
usage control

Takeaways

- Efficient
- Great UX
- Customizable



Takeaways

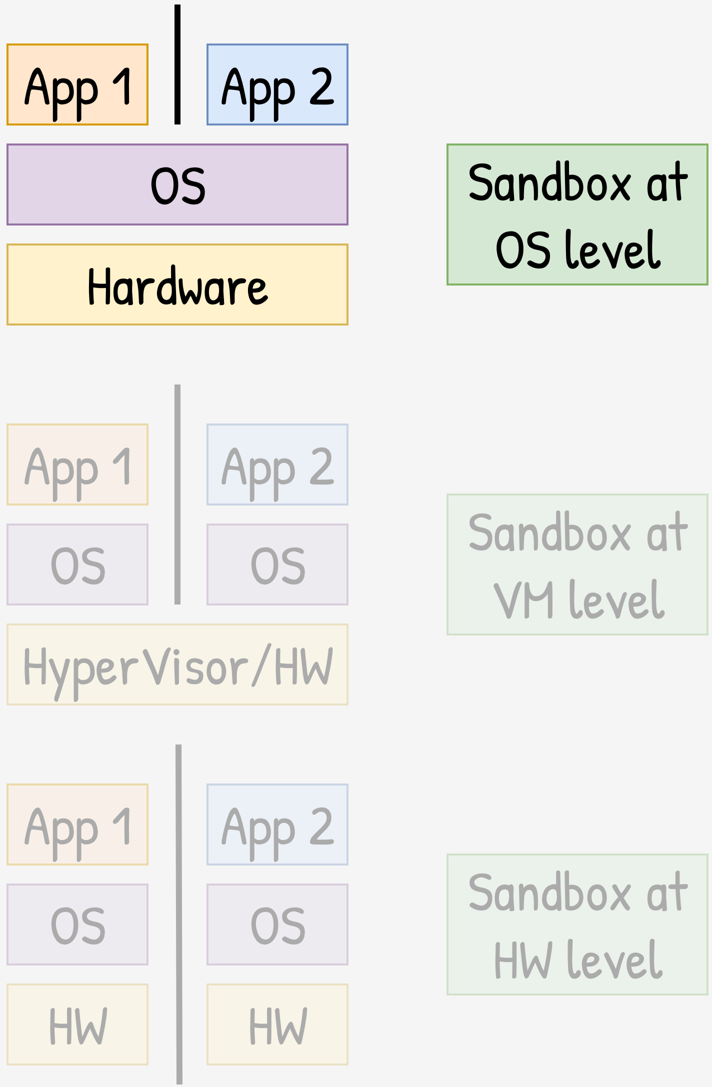
Efficient

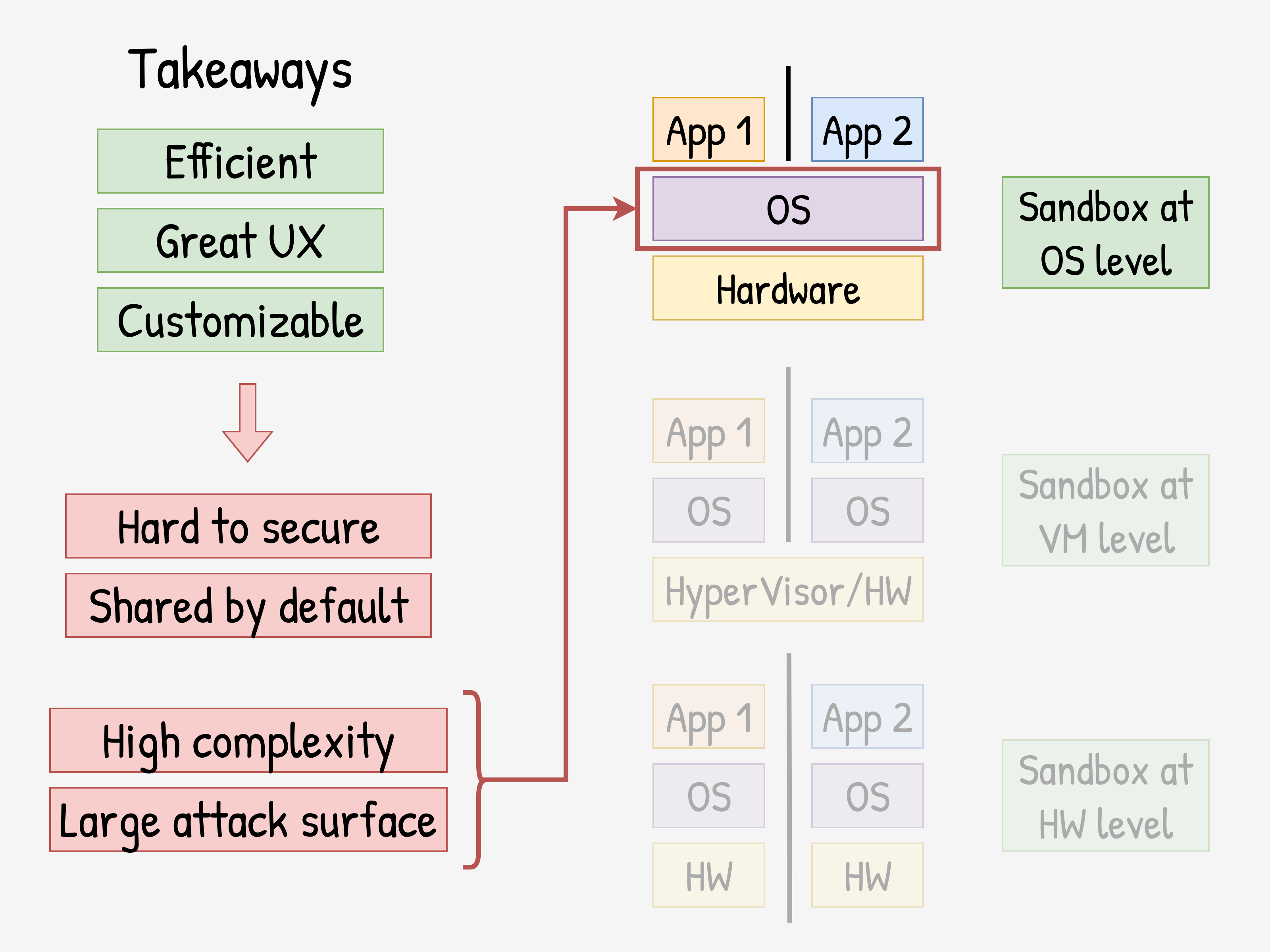
Great UX

Customizable

Hard to secure

Shared by default




Takeaways
Efficient
Great UX
Customizable
Hard to secure
Shared by default
High complexity
Large attack surface
App 1
App 2
OS
Hardware
Sandbox at OS level
App 1
App 2
OS
OS
HyperVisor/HW
Sandbox at VM level
App 1
App 2
OS
OS
HW
HW
Sandbox at HW level